CTU

CZECH TECHNICAL UNIVERSITY IN PRAGUE

F3

Faculty of Electrical Engineering
Department of Computer Science

Bachelor's Thesis

Web-based application for managing private bank accounts

Yelyzaveta Taranova
Software Engineering and Technology

Leden 2013, srpen 2020
Supervisor: Ondřej Vondrouš

Acknowledgement / Declaration

Chtěl bych poděkovat své manželce Ludmile za podporu nejen finanční. Díky tomu mohu na svém pracovišti dělat, co mě baví, a nejsem stresován výplatní páskou.

I declare that I have prepared the submitted thesis independently and that I have cited all the information sources used in accordance with the Methodological Guideline on the observance of ethical principles in the preparation of university final theses.

In Prague 13. 13. 2013 ???

.....

Abstrakt / Abstract

Tento projekt se zaměřuje na návrh a vývoj webové aplikace pro správu osobních bankovních účtů prostřednictvím agregace finančních údajů z více českých bank na jednom místě. Jeho hlavním cílem je poskytnout uživatelům jednotný, přehledný a uživatelsky přívětivý náhled na jejich finance, aniž by museli přepínat mezi různými systémy internetového bankovnictví.

Pro určení struktury a funkčnosti aplikace projekt začíná analýzou stávajících platforem pro správu osobních financí. Tato analýza identifikuje běžné funkce, silné stránky a omezení současných řešení a slouží jako základ pro návrh klíčových funkcí navrhované aplikace.

Klíčová slova: Bankovní aplikace, osobní bankovnictví, centralizace financí, webová aplikace, Spring Boot Framework, REST API, Java, službově orientovaná architektura.

This project focuses on the design and development of a web application for managing personal bank accounts by aggregating financial data from multiple Czech banks in one place. Its primary purpose is to provide users with a unified, clear, and user-friendly overview of their finances without requiring them to switch between different online banking systems.

To define the structure and functionality of the application, the project begins with an analysis of existing personal finance platforms. This analysis identifies common features, strengths, and limitations of current solutions and serves as the basis for designing the core functions of the proposed application.

Keywords: Banking application, personal banking, finance centralization, web application, Spring Boot Framework, REST API, Java, service-oriented architecture.

/ Contents

Chapter 1
Introduction

Keeping track of personal finances is getting harder since most people now use multiple bank accounts and cards from different institutions. Czech banks generally have good online platforms, but checking your overall financial situation usually means logging into several different apps to piece the information together.

Thanks to PSD2 and open banking rules in the EU, it is now possible for third-party apps to access bank data securely, as long as the user gives permission. This thesis takes advantage of that to build a unified personal finance dashboard.

The goal of this project is to develop a web application that uses these modern banking APIs to pull a user's accounts into one place. Through a single interface, users can check their current balances, track transactions across their various Czech bank accounts, and manage their budgets. The application is designed purely for personal use and focuses on organizing data safely. To keep the project secure and focused, it only reads data and strictly avoids active operations like sending payments or changing account settings.

1.1 Motivation

Even though Czech banks support PSD2 APIs, their systems are still isolated from one another. If a user has accounts across Česká spořitelna, Air Bank, and Moneta, they have to jump between different apps just to see how much money they actually have.

There are international apps that try to solve this, but they usually struggle to connect with Czech banks. When they do work, they often rely on third-party data aggregators, which raises privacy concerns and usually means the user's data is passing through extra hands.

Because of this, the project is motivated by four main goals:

- **Centralization** — pulling all accounts and transaction history into a single application with an easy-to-read dashboard.
- **Transparency** — giving the user a clear, instant view of their cash flow without requiring them to do manual calculations.
- **Control and privacy** — keeping the user's financial data secure and private, without relying on third-party data brokers.
- **Simplicity** — building a straightforward personal tool instead of a massive, complicated fintech product.

1.2 Objectives

The main goal of this thesis is to build a web application that safely connects to different Czech banks and helps the user make sense of their money. Specifically, the project focuses on four main tasks:

1. Build a single dashboard for all accounts

Instead of forcing users to log into several different bank apps, the application will display all current balances and transaction histories in one place.

2. Make spending habits easy to understand

Raw lists of transactions are hard to read. The app will categorize expenses and show basic monthly trends so users can quickly see where their money is going without having to use a spreadsheet.

3. Put open banking to practical use

Instead of just conducting theoretical research, the project will actively utilize PSD2 APIs from Czech banks to demonstrate how this technology can safely solve a real-world problem for regular users.

4. Learn from existing applications

Before designing the app, current personal finance tools will be analyzed. Seeing what works—and what frustrates users—in existing apps will directly shape the features and interface of this project.

Chapter 2
Existing Solutions Overview

Before designing the new application, it is important to analyze how existing tools handle personal finance management. The core features across the industry are generally the same: pulling transaction data, categorizing spending, and providing visual budgeting tools. Applications differ significantly in their technical implementation, specifically in how they access banking data and their geographic focus. This section evaluates several popular solutions to identify their strengths, design patterns, and limitations regarding Czech banking systems.

2.1 Budgeting Applications

Applications built for the US market often set the standard for user interface design and budgeting methodology.

- **You Need A Budget (YNAB)** (https://www.ynab.com/) uses a strict budgeting approach where every unit of income is assigned a specific category before it is spent. From a functional perspective, its budgeting logic is highly effective.
- **Monarch Money** (https://www.monarchmoney.com/) focuses on household finances, allowing multiple users to manage a shared database. It also actively scans transaction histories to flag recurring subscriptions.
- **PocketGuard** (https://pocketguard.com/) calculates the user's exact disposable income. The application subtracts scheduled upcoming expenses from the current account balance, providing a direct metric of the funds currently available for discretionary spending.

Limitations: although functionally advanced, these applications share a major technical flaw for European users. They rely almost exclusively on North American API aggregators like Plaid. Because they lack integration with the European PSD2 framework, they cannot connect to Czech banks.

2.2 Wealth Trackers

- **Empower** (https://www.empower.com/) is designed to track entire financial portfolios. It aggregates daily checking accounts alongside investment portfolios, retirement funds, and real estate values. The application's primary function is to calculate total net worth and analyze long-term asset allocation rather than strictly monitoring daily spending.
- **Quicken Simplifi** (https://www.quicken.com/products/simplifi/) connects to bank accounts, credit cards, and investment accounts to provide real-time updates on cash flow and spending trends. Users can create custom budgets, establish savings goals, and track specific spending categories through personalized watchlists.

Limitations: tracking multiple asset classes requires a complex user interface. Both applications share the same geographic data restrictions. They rely entirely on North American financial aggregators and lack direct API support for Central European financial institutions. Consequently, they cannot connect to Czech banks via open banking APIs.

2.3 Global Solutions with European Support

- **Spendee** (https://www.spendee.com/) is an application with Czech origins that maintains a large global user base. It functions properly within the Czech Republic by offering detailed transaction categorization and handling multiple currencies. It can automatically synchronize with local institutions like Česká spořitelna or Air Bank, providing an automated overview of daily finances for regional users.
- **Wallet** (https://budgetbakers.com/) is another globally recognized personal finance application developed in the Czech Republic. Similar to Spendee, it provides advanced transaction categorization, budgeting tools, and multi-currency support. It also features automatic data synchronization with Czech banks.

Limitations: to support thousands of banks worldwide, neither application connects to financial institutions directly. Instead, they rely on third-party data aggregators like Salt Edge. This approach introduces a middleman into the data flow. The user must trust a third-party broker with their financial consent and data, which creates potential privacy and security concerns compared to direct bank connections.

2.4 Summary of the Market Gap

Analyzing these existing solutions highlights a clear gap that this thesis aims to fill. The applications with the best interfaces (YNAB, Monarch) cannot connect to Czech banks. The applications that can connect to local banks (Spendee, Wallet) rely on third-party data brokers, adding a layer of privacy risk.

This justifies the core technical approach of this project: building a straightforward, localized tool that connects directly to Czech bank APIs. By skipping the third-party aggregators, the application can guaranty that the user's data travels strictly between their bank and their own private dashboard.

2.5 Key Features in Existing Finance Applications

Based on the analysis of existing solutions, standard personal finance applications typically implement a common set of core functionalities:

- **Account Aggregation** — Connecting various financial accounts to consolidate balances and transaction histories into a single interface. The standard approach involves automated, periodic data synchronization via APIs or data brokers.
- **Transaction Categorization** — Automatically importing and assigning metadata (categories such as `Groceries` or `Utilities`) to individual transactions.
- **Budget Management** — Creating spending limits per category on a defined cycle, usually monthly.
- **Goal Tracking** — Defining specific financial targets, such as accumulating an emergency fund or reducing specific debts.

- **Recurring Payment Detection** — Analyzing transaction histories to identify predictable, repeating charges like utility bills or software subscriptions. This feature allows users to anticipate upcoming cash flow requirements and identify redundant services.
- **Data Visualization** — Generating charts and graphs to visually represent financial states. Standard outputs include charts for categorical spending, graphs for cash flow trends, and historical net worth progression.
- **Automated Notifications** — Triggering system alerts based on specific account states, such as approaching a budget limit, identifying a critically low balance, or detecting unusual transaction patterns.
- **Security and Synchronization** — Implementing strong encryption and multi-factor authentication to protect sensitive financial data.

Chapter 3
Czech Banking APIs Overview

Modern personal finance applications require secure integrations with banking systems. In the Czech Republic, major banks expose standardized APIs mandated by European regulations. These interfaces allow third-party providers to access account information following explicit user consent. The APIs generally align with the Czech Open Banking Standard or the Berlin Group framework, providing consistent data structures for retrieving account details, balances, and transaction histories. This section outlines the regulatory foundation, technical capabilities, security requirements, and access conditions for these APIs.

3.1 The Payment Services Directive (PSD2)

The revised Payment Services Directive (PSD2) is a European Union regulatory framework that fundamentally changed how financial data is managed. Prior to this legislation, banks held exclusive control over their customers' account information. PSD2 mandates that all European financial institutions must provide secure, standardized API access to licensed Third-Party Providers.

This regulation established two primary types of access: Payment Initiation Services (PIS) and Account Information Services (AIS). For the scope of this application, only AIS is relevant. Under the AIS framework, a user can grant a third-party application explicit consent to retrieve their account information and transaction history in a read-only format. The directive ensures that users maintain complete ownership of their financial data and strictly control who can access it.

3.2 Authentication and Security Protocols

Czech institutions implement these mandated services using RESTful API architectures. Authentication relies strictly on the OAuth 2.0 protocol combined with Strong Customer Authentication (SCA). During the synchronization process, the application redirects the user to the bank's own secure portal. At this stage, the user must complete the SCA process to formally grant access. This authorization flow ensures that the custom application never handles or stores direct login credentials. All subsequent data exchanges require mutual TLS encryption to guarantee transport security.

3.3 Sandbox vs. Production Environments

Accessing banking APIs depends entirely on the deployment environment. Connecting to production APIs to fetch real user data requires a valid eIDAS certificate and an official license from the Czech National Bank to operate as a regulated Account Information Service Provider. Acquiring this regulatory approval is highly complex and generally not feasible for private tools or academic projects.

To resolve this access limitation, financial institutions provide developer sandbox environments. These sandboxes replicate the exact architecture and endpoint structure of the production APIs and return realistic, mock financial data. Developers can register applications, implement the OAuth authentication flow, and test data retrieval processes without holding a formal PSD2 license.

3.4 Comparison of Czech Banking APIs

To successfully integrate multiple financial institutions into a unified application, it is necessary to evaluate the technical capabilities and access requirements of their respective Application Programming Interfaces (APIs). This section provides a comparative analysis of the major banks used in the Czech Republic: Komerční banka, Česká spořitelna, Fio Bank, Air Bank, Raiffeisenbank, ČSOB, Moneta Bank, and UniCredit.

3.4.1 Provided Features

All mentioned financial institutions fully support the core Account Information Services required for a personal finance dashboard. Specifically, their APIs provide standardized endpoints for **listing personal accounts**, **listing current account balances**, and **listing transaction history**.

3.4.2 Production Environment Requirements

To access the production environments and retrieve real user data, all six institutions enforce strict regulatory and technical prerequisites. Every bank requires the applicant to hold a valid PSD2 AISP license. From a technical perspective, establishing a production connection universally requires a Qualified Website Authentication Certificate (QWAC).

3.4.3 Sandbox Environment Requirements

The requirements for sandbox environments are significantly less restrictive. For all of the evaluated financial institutions, developers simply need to complete a standard registration process on their respective developer portals to obtain sandbox credentials, API keys, and test certificates.

All mentioned banks, except Moneta Money Bank, Fio Bank, and UniCredit, allow independent developers and private applications to access their testing infrastructure. Moneta Money Bank, Fio Bank, and UniCredit Bank prohibit academic projects and private applications from using their testing platforms, making their APIs unavailable for independent development and research.

3.5 Conclusion

The analysis of banking APIs available in the Czech Republic shows that major banks offer highly standardized Account Information Services driven by the PSD2 framework. Access to real production data remains strictly regulated and limited to licensed third-party providers. The sandbox environments offered by these banks significantly lower the access barriers. These simulated environments provide a highly suitable platform for development, prototyping, and fully verifying functional architecture. This project will make use of the accessible sandbox APIs: Komerční banka, Česká spořitelna,Air Bank, Raiffeisenbank and ČSOB.

Chapter 4
Business Analysis

This chapter defines the business context of the application and translates user needs into structured functional and non-functional requirements. It identifies the target audience, describes expected system behavior, and outlines key business processes supported by the application. The goal of this analysis is to ensure that the proposed solution aligns with real user needs while remaining technically feasible and compliant with regulatory constraints.

4.1 Functional Requirements

This section specifies the functional requirements of the application, defining what the system shall do from the user's perspective. The requirements are grouped by functional areas and cover all major interactions, including authentication, bank integration, financial data retrieval, visualization, budgeting, and administration.

4.1.1 Access to Public Pages

ID	Title	Description
FR1	Displaying the landing page	The system shall allow anonymous users to view the landing page with a general description of the application and its purpose, information about features, security, and supported banks.

Table 4.1. Functional Requirements — Access to Public Pages

4.1.2 Profile Management

ID	Title	Description
FR2	Registration	The system shall allow anonymous users to create an account via the registration form.
FR3	Log in	The system shall support logging in using an email and password.
FR4	Log out	The system shall allow the user to log out.

Table 4.2. Functional Requirements — Profile Management

4.1.3 Bank Integration

ID	Title	Description
FR5	Linking a bank	The system shall allow users to link a bank from defined list of financial institutions
FR6	Token storage	The system shall securely store tokens required for retrieving data.
FR7	Bank disconnection	The system shall allow users to disconnect any previously linked bank.

Table 4.3. Functional Requirements — Bank Integration

4.1.4 Financial Data Retrieval

ID	Title	Description
FR8	Product retrieval	The system shall retrieve customers products from all linked banks.
FR9	Account balances retrieval	The system shall retrieve the balance for each account from all linked banks.
FR10	Transaction history retrieval	The system shall retrieve the transaction history for each account from all linked banks for the specified time period.
FR11	Financial data caching	The system shall cache financial data periodically.

Table 4.4. Functional Requirements — Financial Data Retrieval

4.1.5 Visualization

ID	Title	Description
FR12	Displaying a dashboard	The system shall display a consolidated dashboard containing an overview of assets and liabilities.
FR13	Displaying statistics	The system shall present graphs of spending and income for the selected time period.
FR14	Displaying transactions	The system shall display transactions in a clear list format for every account from all linked banks.
FR15	Displaying products	The system shall display products with balances in a clear list format for every account from all linked banks.
FR16	Displaying budgets	The system shall display budgets in a clear list format.
FR17	Displaying profile	The system shall display the profile with user data.

Table 4.5. Functional Requirements — Visualization

4.1.6 Categorization

ID	Title	Description
FR18	Transactions categorization	The system shall automatically categorize transactions using predefined rules.
FR19	Products categorization	The system shall automatically categorize products using predefined rules.

Table 4.6. Functional Requirements — Categorization

4.1.7 Transactions Grouping

ID	Title	Description
FR20	Transactions Grouping	The system shall group transactions by month and by category.

Table 4.7. Functional Requirements — Transaction Grouping

4.1.8 Product Filtering

ID	Title	Description
FR21	Product Filtering	The system shall allow the filtering of products by bank.

Table 4.8. Functional Requirements — Product Filtering

4.1.9 Budgeting

ID	Title	Description
FR22	Setting monthly budgets	The system shall allow users to set monthly budgets for categories.
FR23	Budget warnings	The system shall display warnings or highlights when a budget is exceeded.

Table 4.9. Functional Requirements — Budgeting

4.1.10 System Monitoring

ID	Title	Description
FR24	System logs	The system shall provide logs for authentication attempts, API errors, and system faults.

Table 4.10. Functional Requirements — System monitoring

4.2 Non-Functional Requirements

ID	Title	Description
NFR1	Security	The application must use secure storage and must not store sensitive data in an open form.
NFR2	Resilient system	The system must handle API failures gracefully.
NFR3	Application performance	The application should load the dashboard within 20 seconds.
NFR4	Browser compatibility	The application must function properly on current versions of major web browsers.

Table 4.11. Non-functional Requirements

4.3 Use cases

This section presents the main use cases of the system, describing how different types of users interact with the application. Use cases are divided according to user roles and illustrate the scope of functionality available to each role. The diagrams provide a high-level overview of system behavior and support the understanding of functional requirements.

4.3.1 Anonymous User

Anonymous users are visitors who access the application without creating an account or logging in.

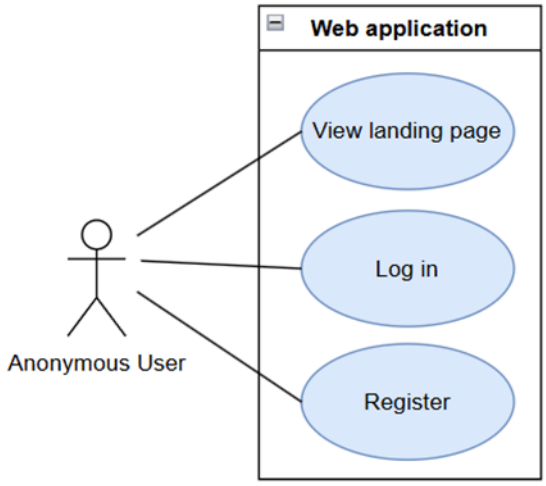

Figure 4.1. Use Cases — Anonymous User

This diagram illustrates the initial interaction points available to an unauthenticated individual, designated as the `Anonymous User` actor. Within the system boundary of the web application, this actor has direct associations with three primary use cases: viewing the public-facing landing page, creating a new account, and authenticating into the system.

These three use cases directly implement the foundational access and authentication functional requirements defined in section 4.1:

- **View landing page** — This use case implements FR1 (Displaying the landing page) from section 4.1.1. It ensures the system allows unauthenticated users to read general information regarding the application's purpose, features, and security.
- **Register** — This use case implements FR2 (Registration) from section 4.1.2. It represents the workflow where an anonymous user submits their credentials via a registration form to create a new profile and gain system access.
- **Log in** — This use case implements FR3 (Log in) from section 4.1.2. It fulfills the requirement for the system to authenticate a user via their credentials, successfully transitioning their state from an Anonymous User to a Logged-in User.

4.3.2 Logged-in User

A logged-in user has access to the core features of the application, including connecting banking accounts and viewing financial data.

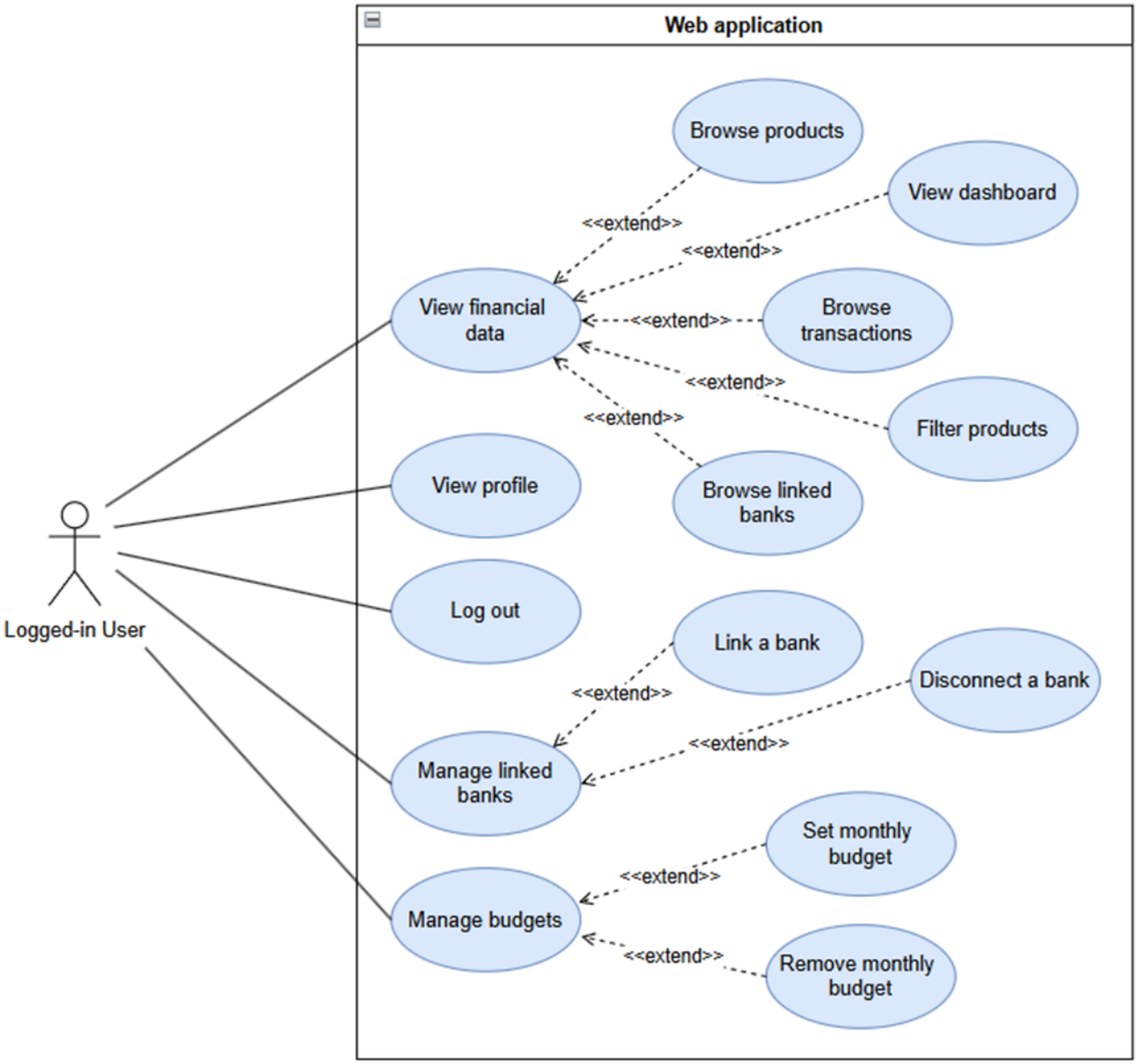

Figure 4.2. Use Cases — Logged-in User

This diagram illustrates the set of interaction scenarios available to a user who has successfully authenticated into the system, designated as the `Logged-in User` actor. Within the web application boundary, this actor maintains direct associations with five primary use cases. These base use cases are further modified through the specific extension relationship `<<extend>>` to encompass the full suite of functional requirements.

The primary use cases and their corresponding extensions implement the system's core capabilities:

- **View financial data** — This use case serves as the central hub for financial visualization and implements functional requirements from section 4.1.5. It is extended by specialized actions allowing the user to view the dashboard (FR12), browse transactions (FR14), browse products (FR15), browse linked banks, and filter products (FR21).
- **View profile** — This use case implements FR17 from section 4.1.5, enabling the user to access and review their personal account information.

- **Log out** — This use case implements FR4 from section 4.1.2, fulfilling the requirement to securely terminate the active user session.
- **Manage linked banks** — This use case represents the primary interface for banking integration and implements functional requirements from section 4.1.3. It is extended by specific actions to link a bank (FR5) and disconnect a bank (FR7).
- **Manage budgets** — This use case represents the financial planning functionalities and implements functional requirements from section 4.1.9. It is extended by specific actions to set the monthly budget (FR22) and remove the monthly budget.

4.4 Process Analysis

Process analysis describes the key workflows supported by the application and explains how individual components interact during typical operations. These processes focus on user interactions with external banking systems, internal data processing, and system responses. The aim is to clarify how functional requirements are realized in practice and to identify critical integration and security steps.

4.4.1 Connecting a Bank Account

The process of connecting a bank account enables the user to grant the application access to their financial data through an external banking API.

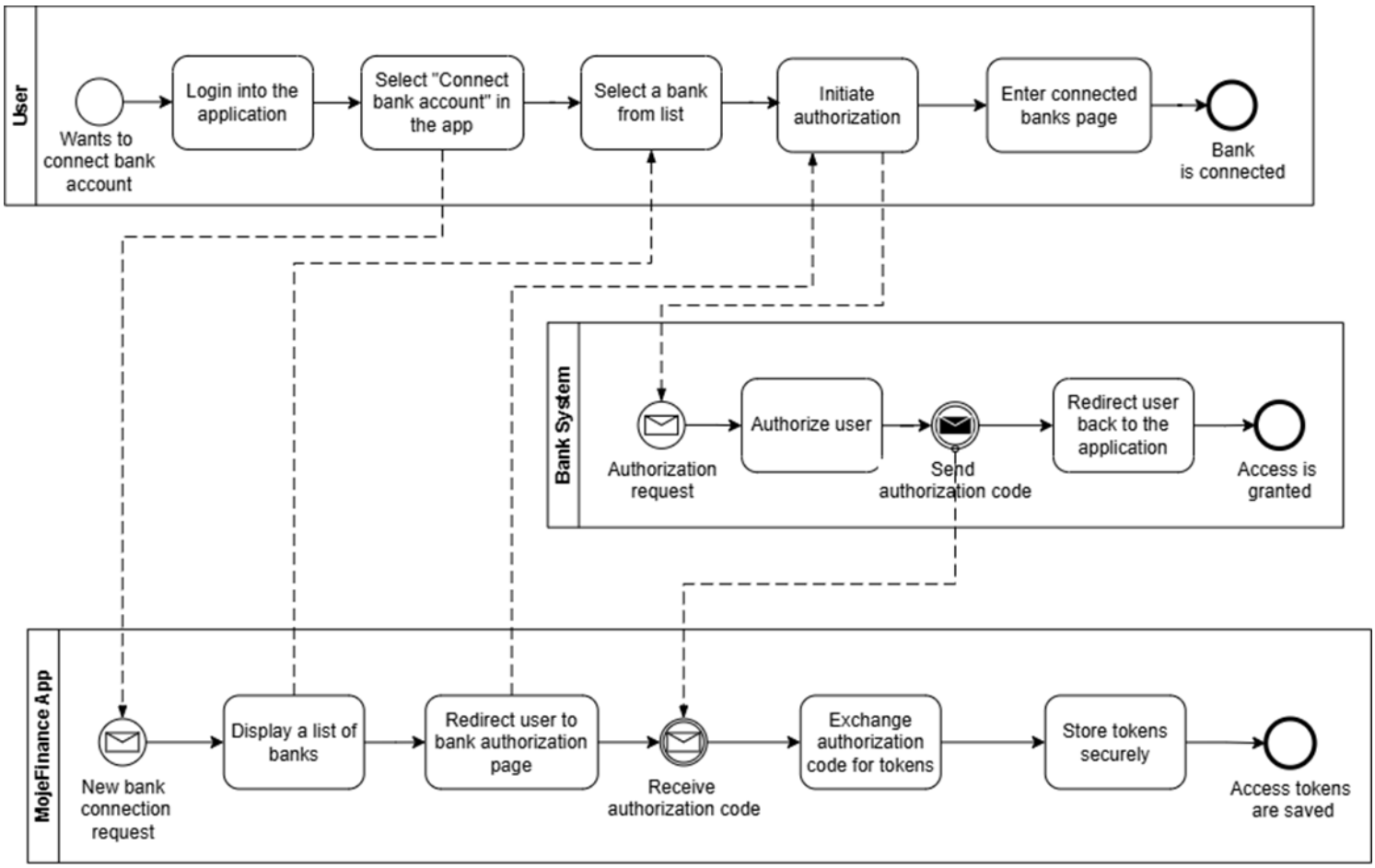

Figure 4.3. Process Diagram — Connecting a Bank Account

The process begins when a logged-in user selects a bank from the list of supported institutions within the application. After initiating the connection, the backend redirects the user to the selected bank's authentication page. At this stage, the user authenticates directly with their bank using the bank's own security mechanisms, such as two-factor authentication or biometric verification. The application itself never accesses or stores the user's banking credentials.

Once authentication is completed, the bank presents a consent screen where the user explicitly approves access to specific data scopes, typically including account information, balances, and transaction history. After consent is granted, the bank redirects the user back to the application along with an authorization code.

The backend service then exchanges this authorization code for an access token using a secure OAuth2 communication channel. The obtained tokens are securely stored in encrypted form and associated with the user's bank connection. The application uses these tokens to retrieve account and transaction data via the bank's API.

Following successful authorization, the system performs an initial synchronization, during which it retrieves the list of available accounts and their associated data. Retrieved accounts are normalized into the internal domain model and cached. Accounts become visible to the user in the application dashboard.

Chapter 5
Technology Analysis

This chapter analyzes the technologies selected for the development of the web application. The goal is to justify the chosen technology stack based on performance, maintainability, scalability, integration capabilities, and long-term sustainability.

5.1 Backend Development

5.1.1 Framework

The **Spring Boot** framework offers standardized approaches to dependency injection, configuration management, and modular application design. While lightweight micro-frameworks like Express.js or Flask offer faster initial setup times, Spring Boot excels in complex enterprise environments. For a financial application handling sensitive user data, strict typing and long-term maintainability over rapid prototyping are essential.

5.1.2 Programming Language

Java was chosen as the primary programming language. In contrast to Python or JavaScript, Java provides strong static typing and relies on the highly optimized Java Virtual Machine. The extensive Java ecosystem offers mature, enterprise-grade libraries for security, cryptography, and complex external API integrations, making it exceptionally well-suited for building a reliable and secure banking system.

5.1.3 Database

PostgreSQL was selected as the primary relational database management system. A personal finance application requires strict structural consistency. PostgreSQL provides the necessary relational integrity to ensure that financial data remains perfectly organized during complex multi-table operations.

5.1.4 Caching

Redis serves as the in-memory data store to accelerate access to frequently requested information. Relying solely on live API calls creates an unacceptable user experience. Compared to simpler local caching solutions like Memcached, Redis supports complex data structures and allows for highly precise control over cache expiration policies. Storing cached values reduces external API calls and minimizes database read operations.

5.1.5 Resilience

Integrating multiple third-party banking APIs introduces a high risk of network failures. Standard error handling is insufficient for external service outages. **Resilience4j** is integrated to implement robust fault-tolerance mechanisms, replacing older, deprecated libraries like Netflix Hystrix. It provides essential patterns such as circuit breakers, automated retries, and rate limiting. By isolating failing external services and preventing cascading system failures, Resilience4j ensures the primary application remains responsive during temporary outages of specific bank providers.

5.1.6 Messaging Configuration

Feign Client ensures communication with external banking APIs. Interacting with external endpoints traditionally requires writing repetitive boilerplate code using standard HTTP clients like RestTemplate or WebClient. Feign resolves this complexity by acting as a declarative web service client. This approach significantly simplifies the codebase, making it highly maintainable when connecting to numerous financial institutions.

5.1.7 Conclusion

The selected backend stack establishes a secure, reliable, and highly scalable architecture perfectly suited for the strict demands of a financial application. In addition to their proven technical capabilities, these specific technologies were chosen due to the author's prior practical experience with them. This existing proficiency ensures an efficient development lifecycle and guarantees the correct implementation of the required security protocols, data integrity measures, and system resilience patterns.

Selected technologies are summarized in the following table.

Category	Technology
Framework	Spring Boot
Programming Language	Java
Database	PostgreSQL
Caching	Redis
Resilience	Resilience4j
Messaging Configuration	Feign Client

Table 5.1. Selected technologies for application backend development

5.2 Frontend Development

5.2.1 Framework

React is a JavaScript library for building user interfaces, created and maintained by Meta (Facebook). It is currently one of the most popular and widely adopted frontend technologies worldwide. React organizes the interface into reusable components (buttons, forms, graphs, cards, charts). This is ideal for finance dashboards, which require modular widgets. Additionally, React uses a virtual DOM, which makes UI updates fast when data changes frequently. Moreover, it has a massive ecosystem of libraries for routing, state management, charts, authentication, and UI styling, which dramatically reduces implementation effort.

Chapter 6
System Design

This chapter describes the overall system structure, architectural decisions, and internal organization of the application. It explains how individual services are arranged, how data flows through the system, and how external banking APIs are integrated. The design focuses on modularity, scalability, and maintainability while respecting security and regulatory requirements.

6.1 Architecture

This chapter presents the architectural design of the application, serving as the foundational blueprint for the entire system. Establishing a robust and well-defined architecture is crucial for ensuring the application's scalability, maintainability, and security — especially given the sensitive nature of financial data and the requirement to integrate with multiple third-party banking APIs securely. The following sections detail the chosen architectural style, the high-level system components, the internal structuring of individual services, and the specific boundaries that separate internal logic from external integrations.

6.1.1 Architectural Style

The development of the application is based on Service-Oriented Architecture (SOA). SOA is an architectural style in which a system is built as a collection of modular, reusable, and network-accessible services. Each service represents a distinct business capability — such as user management, bank integration, data synchronization, or reporting — and exposes its functionality through standardized interfaces, typically REST endpoints.

Although SOA and microservices share conceptual similarities — such as modularization, service boundaries, and network-based communication — they differ significantly in granularity, autonomy, communication style, governance, and deployment. In SOA, services can be independent, but they often share centralized components, such as databases, message busses, or identity providers. In microservices, each service is fully autonomous, owning its own database, technology stack, and deployment pipeline. This eliminates shared dependencies but increases operational complexity.

For this application, SOA provides an optimal balance. It allows for logical separation of domains without the heavy operational overhead of managing distributed databases and complex deployment pipelines required by pure microservices. As the system evolves and the need for higher scalability or independent deployment grows, this SOA foundation can naturally be refined into a microservices architecture.

6.1.2 Architecture Diagram

The architecture diagram illustrates the high-level structure of the application, showing the interaction between the frontend, backend services, external banking APIs, and the database.

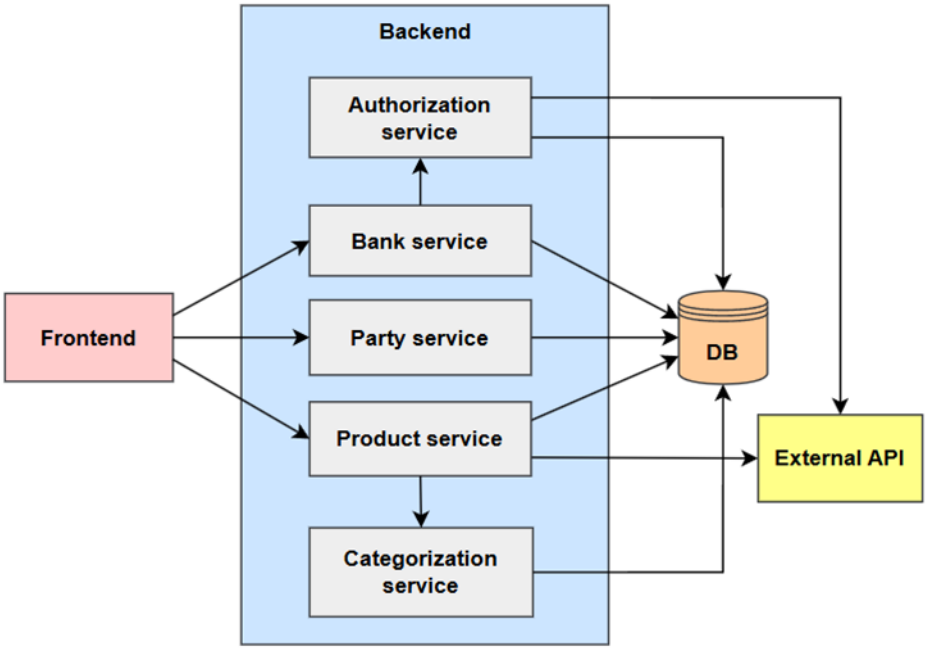

Figure 6.1. Application Architecture

The diagram in Figure 6.1 illustrates the Service-Oriented Architecture adopted for the system. The Frontend acts as the client layer, initiating requests to the backend. The Backend is decomposed into five distinct functional services: the Authorization service, the Bank service, the Party service, the Product service, and the Categorization service. Unlike a pure microservices architecture, where each service strictly owns its own database, these services share a central Database for persistence, ensuring data consistency while keeping the infrastructure manageable. The Product service also acts as a gateway to External APIs, handling the retrieval of banking data.

6.1.3 Internal Architecture

To ensure separation of concerns and maintainability, each service in the system follows a strict layered architecture composed of three main layers:

1. **Presentation Layer** — this layer exposes REST endpoints that allow clients to interact with the system. It handles incoming HTTP requests, validates input structures, and delegates execution to the domain layer. The presentation layer is strictly stateless and does not contain any business logic.
2. **Domain Layer** — this layer encapsulates the core business logic and workflows. It orchestrates operations between repositories, external providers, and domain objects. It is responsible for authorization checks, data transformation, validation rules, and transaction handling. It also contains domain entities and value objects representing the system's primary business concepts.
3. **Messaging and Persistence Layer** — this layer forms the boundary between the internal business logic and external infrastructure. The Messaging component acts as an adapter for external API implementations. Since each bank may provide different endpoints, data formats, authentication flows, or rate limits, it is essential to centralize and standardize these interactions here.

In addition to external messaging, this bottom layer manages data persistence using JPA and Hibernate. It abstracts database operations and provides a clean interface for

the Domain Layer to retrieve and store domain entities without needing to understand the underlying SQL dialects or table structures.

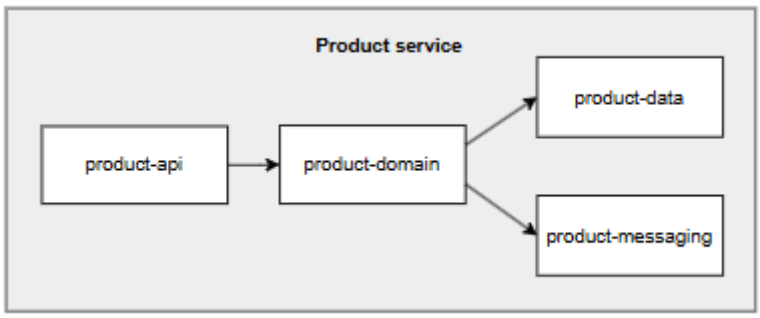

Figure 6.2. Internal Architecture

The internal architecture of the backend services, exemplified here in Figure 6.2 by the Product service, follows this strict layered pattern. The product-api component serves as the Presentation Layer, exposing REST endpoints to the client. This layer passes requests to the product-domain, which encapsulates the business logic and domain entities. The domain layer interacts with two downstream components: product-data, which handles persistence via the database, and product-messaging, which manages integrations with external banking systems.

6.1.4 Component Diagram

The component diagram illustrates the internal structure of the backend system and its interaction with the frontend, external bank APIs, and the database. It highlights how the conceptual layers described above are realized as actual software components.

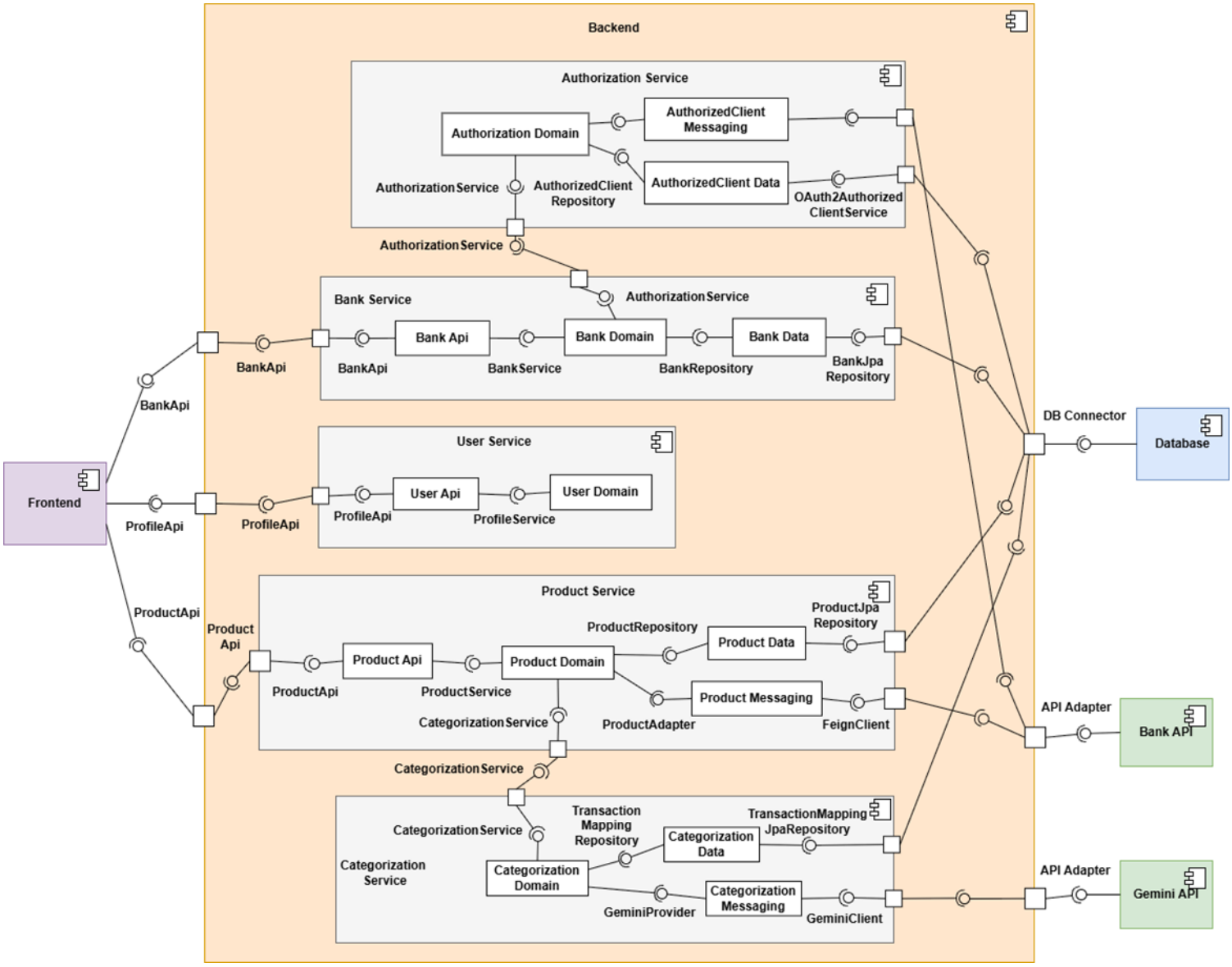

Figure 6.3. Component Diagram

The frontend communicates with the backend through exposed REST APIs. Each request is routed to the appropriate service via dedicated controllers, which represent the entry point to the backend. The diagram shows the four main backend services: Authorization Service, Bank Service, User Service, and Product Service. Each service is isolated within its own boundary, emphasizing modularity and preventing business logic from bleeding across domains.

Within each service, the standard layered architecture is applied. The controller layer handles incoming HTTP requests and forwards them to the application API layer. The API layer defines service-specific interfaces and orchestrates calls to the domain layer.

Persistence responsibilities are delegated to repository and data components (Authorization Data, Bank Data, Product Data), which manage the physical database transactions. Communication with external banking systems is handled exclusively through adapter components. These APIs adapters provide a standardized interface for interacting with varying bank APIs, effectively shielding internal services from external changes, authentication complexities, and network failures.

Overall, the component diagram demonstrates a clean separation between the presentation, application, domain, and infrastructure layers. It highlights the loose coupling between services, clear responsibility boundaries, and the high extensibility of the system.

To provide further context on the system's modularity, the core business responsibilities delegated to each backend service are defined as follows:

- **Authorization Service** — acts as the dedicated security backbone and token vault for all external banking integrations. It centrally manages OAuth2 client registrations, secure token acquisition, and the entire cryptographic lifecycle. By handling the storage, retrieval, and automated refreshing of access and refresh tokens, it ensures that sensitive credentials are strictly isolated from the rest of the application's business logic.
- **User Service** — functions as the centralized user lifecycle manager. It completely isolates user identity and profile management from the financial domains.
- **Bank Connection Service** — manages the logical state of connected banking institutions. It abstracts the highly variable integration details of different Czech banks into a unified internal interface. Crucially, this service does not directly handle or store any security tokens. Instead, it delegates all token retrieval and refresh operations to the Authorization Service, allowing it to focus purely on orchestrating external bank communication and metadata.
- **Product Service** — serves as the core financial engine and data aggregation layer of the application. It is responsible for orchestrating financial data retrieval and normalizing data structures from various external banks into a single domain model.
- **Categorization Service** — is responsible for categorizing transactions and products based on the incoming information from the external bank API.

6.2 Data design

The data design defines how information is structured, stored, and processed within the application. It focuses on the consistency, integrity, and normalization of financial data retrieved from external APIs, as well as manually created fictive data. The design ensures that real and fictive accounts can coexist within a unified data model, providing a seamless experience for the user, regardless of the data source.

6.2.1 Database Model

The database model describes the relational structure used to persist application data, user profiles, and external integration metadata.

6.2.1.1 Authorized Client Table

OAuth2 authorized client	
PK	client_registration_id: varchar(100)
PK	principal_name: varchar(200)
	access_token_type: varchar(100)
	access_token_value: bytea
	access_token_issued_at: timestamp
	access_token_expires_at: timestamp
	access_token_scopes: varchar(1000)
	refresh_token_value: bytea
	refresh_token_issued_at: timestamp
	created_at: timestamp

Figure 6.4. Authorized Client Table

The `OAuth2 authorized client` table relies on a standardized schema utilized by Spring's `OAuth2AuthorizedClientService`. Its primary responsibility is to securely persist authorized client instances across application restarts. This includes storing the `access_token_value`, `refresh_token_value`, their respective expiration timestamps, and granted scopes. By delegating this to Spring Security, the application ensures that sensitive token lifecycle management (such as automatic token refreshing) is handled safely.

6.2.1.2 Bank Connection Table

Bank connection	
PK	principal_name: varchar(200)
PK	client_registration_id: varchar(100)
	bank_name: varchar(50)
	bank_connection_status: varchar(50)
	manually_created: boolean

Figure 6.5. Bank Connection Table

`Bank connection` table, detailed in Figure 6.5, stores application-specific data about a user's linked bank. It utilizes a composite primary key — `principal_name` (representing the user) and `client_registration_id` (representing the specific bank provider).

It tracks the `bank_name`, the current `bank_connection_status`, and a `manually_created` flag.

6.2.1.3 Budget Table

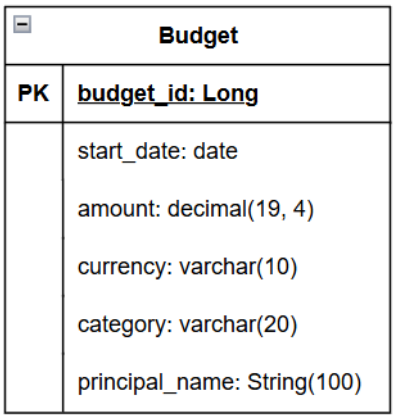

Figure 6.6. Budget Table

The Budget entity, presented in Figure 6.6, manages customer budgets. It is designed to store the financial limits users establish for their spending. The **start_date** field represents a recurring date when the budget is restarted. The defined financial limit is securely stored in the **amount** field. The currency code associated with this allocated amount, such as CZK or EUR, is specified within the **currency** field. The **category** attribute defines the specific spending group to which the limit applies. Finally, the **principal_name** attribute acts as a field that stores the unique identifier of the authenticated user who owns the budget.

6.2.1.4 Categorization Tables

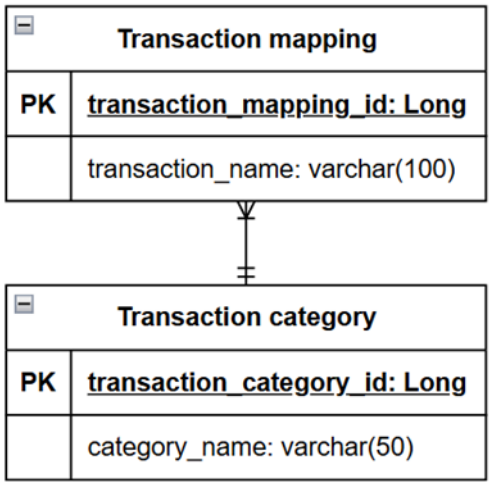

Figure 6.7. Categorization Tables

The `Transaction category` table defines the standardized categories available within the system. It contains a primary key, `transaction_category_id`, and the `category_name` representing the classification (e.g., Groceries, Salary).

The `Transaction mapping` table is responsible for linking specific raw transaction information encountered in banking data (stored in the `transaction_name` field) to

these standardized categories. It uses `transaction_mapping_id` as its primary key. The schema establishes a many-to-one relationship between `Transaction mapping` and `Transaction category`, allowing multiple distinct transaction names from various bank records to be cleanly grouped under a single unified category.

6.2.2 Domain Class Model

The domain class model represents the core business entities and their relationships within the application. It abstracts technical concerns and focuses on financial concepts such as users, bank connections, accounts, and transactions. This model serves as the foundation for business logic implementation and aligns with the layered architecture used in the system.

6.2.2.1 User Service

This section outlines the object-oriented representation of user profiles within the system. The model isolates the properties required to identify, authenticate, and manage the privileges of individuals using the platform.

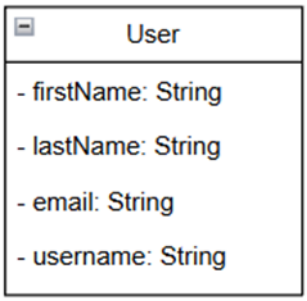

Figure 6.8. Domain Class Model - User Service

The User Service domain model focuses on user management. The primary class, `User`, contains personal attributes such as first name, last name, username and email.

6.2.2.2 Bank Connection Service

The class model for the Bank Connection Service maps the connection metadata required to manage integrations with third-party banking institutions.

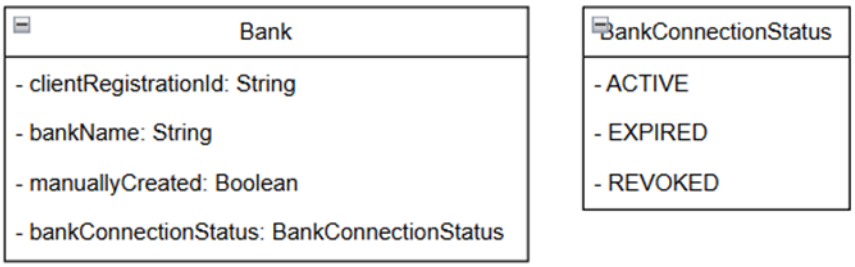

Figure 6.9. Domain Class Model - Bank Connection Service

This domain model handles the metadata and state of banking integrations rather than the security credentials. The primary entity, `Bank`, identifies the institution using a `clientRegistrationId` and records the `bankName`. It also utilizes a `manuallyCreated` flag to seamlessly distinguish between automated integrations and user-defined banks. The lifecycle of the integration is managed by the `bankConnectionStatus` attribute, utilizing the `BankConnectionStatus` enum to track if a connection is currently ACTIVE, EXPIRED, or REVOKED.

6.2.2.3 Product Service

The Product Service domain model represents the most data-intensive part of the application. It defines the relations between financial products, individual transactions, data aggregations, and user-defined budgets.

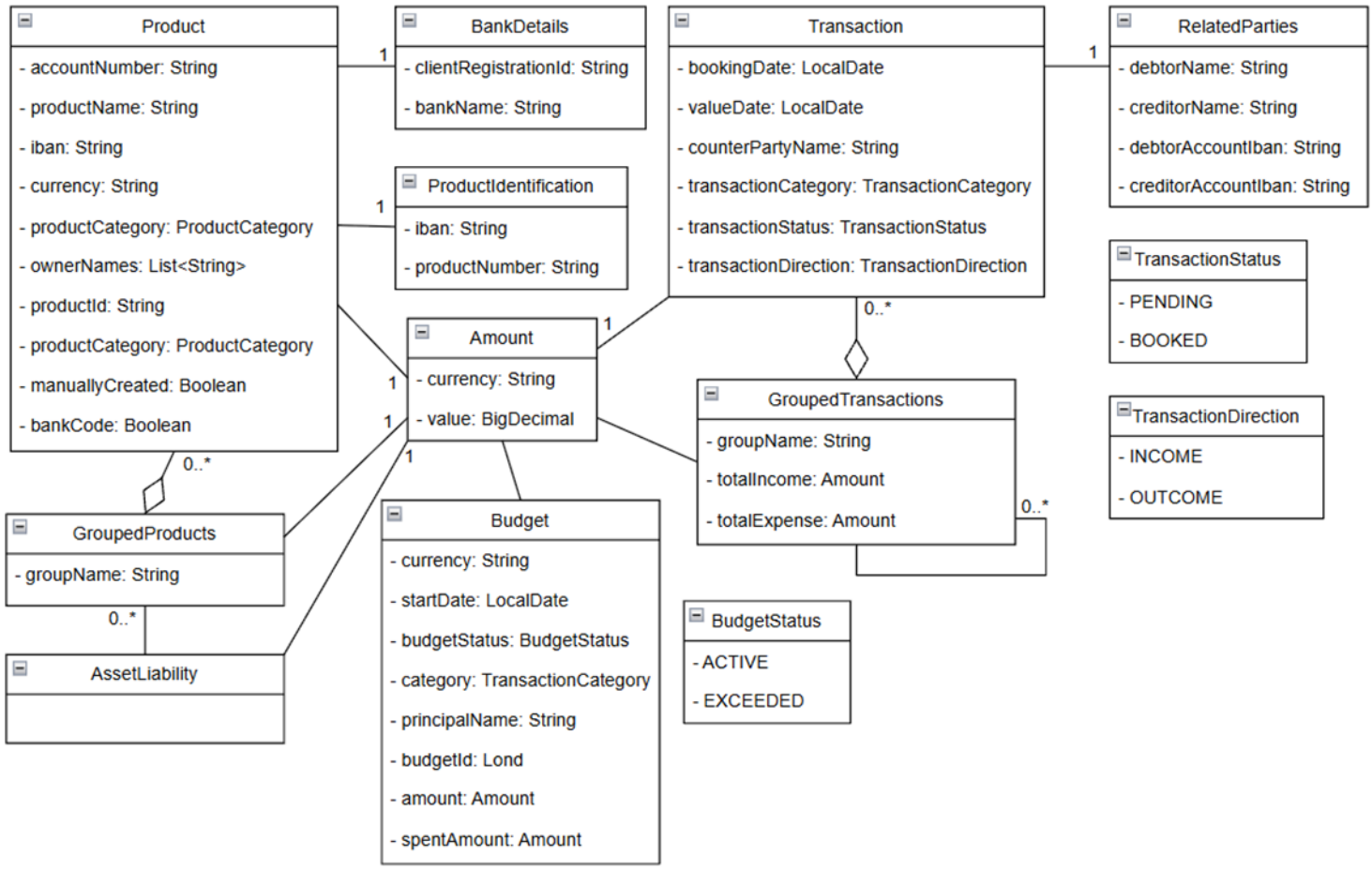

Figure 6.10. Domain Class Model - Product Service

The Product Service model forms the core financial logic of the system. It is centered around the `Product` entity, which serves as a unified representation for bank accounts.

Individual financial movements are recorded by the `Transaction` entity. This class captures core details such as booking dates, processing states (`TransactionStatus`), and cash flow direction (`TransactionDirection`). Complex counterpart details (debtor and creditor information) are extracted into a dedicated `RelatedParties` entity.

To directly support frontend analytics and dashboard visualizations, the model replaces specific time-based or category-based classes with versatile composition structures: `GroupedTransactions` and `GroupedProducts`. These entities logically organize raw financial records for efficient reporting. Additionally, the `AssetLiability` class connects to these grouped products to facilitate high-level financial summaries.

The `Budget` entity enables goal-oriented financial planning, allowing users to set monetary targets based on a specific category and track progress via the `BudgetStatus`. Finally, to ensure strict architectural consistency across the application, the classification enumerations utilized within this model, specifically `ProductCategory` and `TransactionCategory`, are located and maintained within the Shared Service module.

6.2.2.4 Shared Service

The Shared Service module functions as a foundational, cross-cutting component within the application architecture. Its primary objective is to centralize common programming logic, thereby eliminating code redundancy and ensuring strict systemic consistency across all independent functional domains.

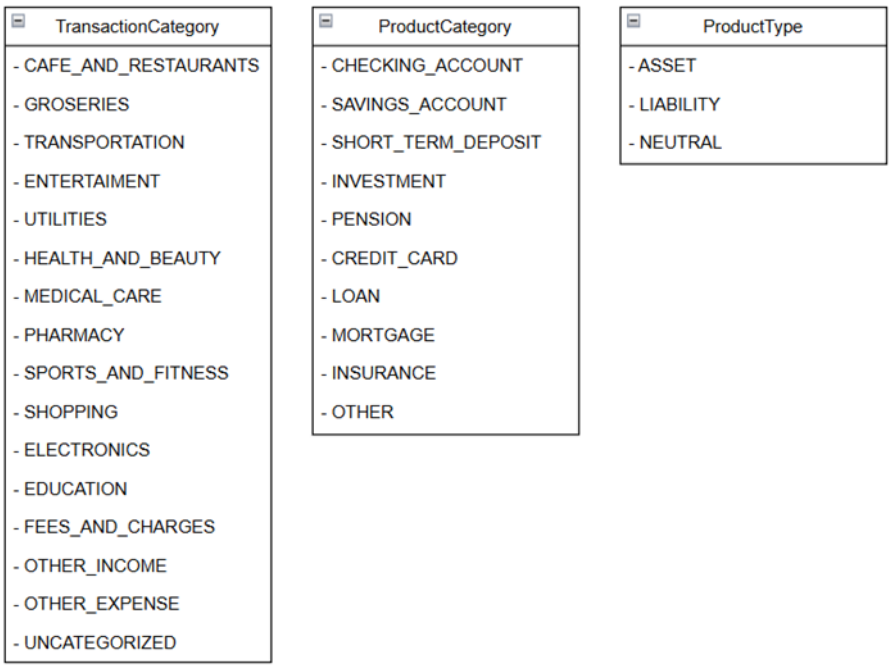

Figure 6.11. Domain Class Model - Shared Service

The Shared Service model, illustrated in Figure 6.11, acts as a central model for the application's core enumerations. By isolating these classification structures into a dedicated module, the system ensures strict data consistency and eliminates code redundancy across all other functional domains.

The model defines three primary enumerations. The `TransactionCategory` enumeration provides a set of predefined labels used to automatically classify individual transactions. This structure includes everyday expense groups such as groceries, transportation, and entertainment, alongside broader classifications for income, fees, and uncategorized records.

Similarly, the `ProductCategory` enumeration standardizes the classification of all connected financial accounts. It encompasses standard banking products, including checking accounts and savings accounts, as well as complex financial instruments like investments, mortgages, and pensions.

Finally, the `ProductType` enumeration introduces a high-level financial grouping by categorizing products as an asset, a liability, or a neutral entity. This specific distinction provides the fundamental logic required for the accurate calculation and visualization of the user's overall net worth within the consolidated dashboard.

Chapter 7
Implementation

This chapter details the practical realization of the personal finance system, translating the previously defined architectural concepts and data models into a fully functional backend application. The development process is structured into five logical phases, reflecting the requirements of building a secure and robust enterprise system. The chapter begins by outlining the security infrastructure; subsequent sections detail the realization of core business features, the application of caching and resilience patterns to guarantee system stability, and the containerized deployment strategy. Finally, the chapter describes the centralized error-handling architecture.

7.1 Frontend Implementation

The primary objective of this project is the design and implementation of a robust backend architecture for financial data aggregation. Consequently, the frontend application serves primarily as a visual interface to demonstrate the backend capabilities and present the aggregated results to the user in a readable format. Due to project time constraints and the author's core expertise residing in backend development, the frontend layer was implemented with the assistance of Artificial Intelligence tools.

The client-side application is constructed using the React framework and bundled with Vite. It relies on standard ecosystem libraries, including React Router for client-side navigation and Axios for managing asynchronous HTTP communications.

To interact securely with the backend services, the frontend implements a centralized `AuthContext`. This context dictates the client-side authentication state and manages the Keycloak OAuth 2.0 lifecycle. It handles the initial login redirects, securely persists the access and refresh tokens within the browser's local storage, and orchestrates automatic, silent token rotation in the background before the short-lived access tokens expire.

Network communication is standardized through a custom Axios client configuration defined within `axiosClient.js`. Request interceptors automatically inject the current Bearer token into the authorization headers of all outgoing API calls. The application also defines dedicated routing components, such as the `BankCallback` page, to successfully intercept, parse, and process asynchronous redirection flows from external banking portals.

The user interface is divided into modular, domain-specific views, including the dashboard, transaction ledger, active bank connections, and budgeting interfaces. These components utilize custom React hooks (such as `useFetchConnectedBanks` and `useCashFlowAnalytics`) to consume the backend REST endpoints, aggregate the JSON payloads, and render real-time financial overviews without containing complex business logic themselves.

7.2 Security

Securing user identities and protecting sensitive financial data are the foundational requirements of the implemented architecture. The system adopts a centralized, delegated authentication model to isolate security concerns from standard business logic.

7.2.1 Keycloak

Keycloak is used as the centralized Identity and Access Management server. It handles the complete lifecycle of user authentication, independently managing user registrations, data, and securely storing credentials. By delegating these responsibilities entirely to Keycloak, the primary application backend is relieved of directly processing sensitive passwords and user data. The administrative configuration, including realm settings, token lifespans, and security policies, is provisioned through a predefined realm export file.

To facilitate the secure authorization process, the Keycloak realm defines two distinct clients corresponding to the application's architectural layers:

- **mojefinance-frontend** — This client is configured as a public client, designed for the user-facing web application. It is responsible for initiating the standard OAuth 2.0 authentication flow. When a user attempts to log in, the frontend client redirects them to Keycloak's hosted authentication pages. Upon successful credential verification, this client receives the resulting JSON Web Tokens, specifically the access token and refresh token, which are subsequently used to authorize requests to the server.
- **Backend client** — This client is configured as a bearer-only client, designed to protect the system's internal REST API endpoints. It does not actively initiate login flows. Instead, when the frontend transmits an HTTP request containing an access token in the authorization header, the backend client intercepts the transmission. It cryptographically verifies the token's signature using Keycloak's public keys, checks the expiration status, and evaluates the embedded claims to enforce strict access control policies before executing any business logic.

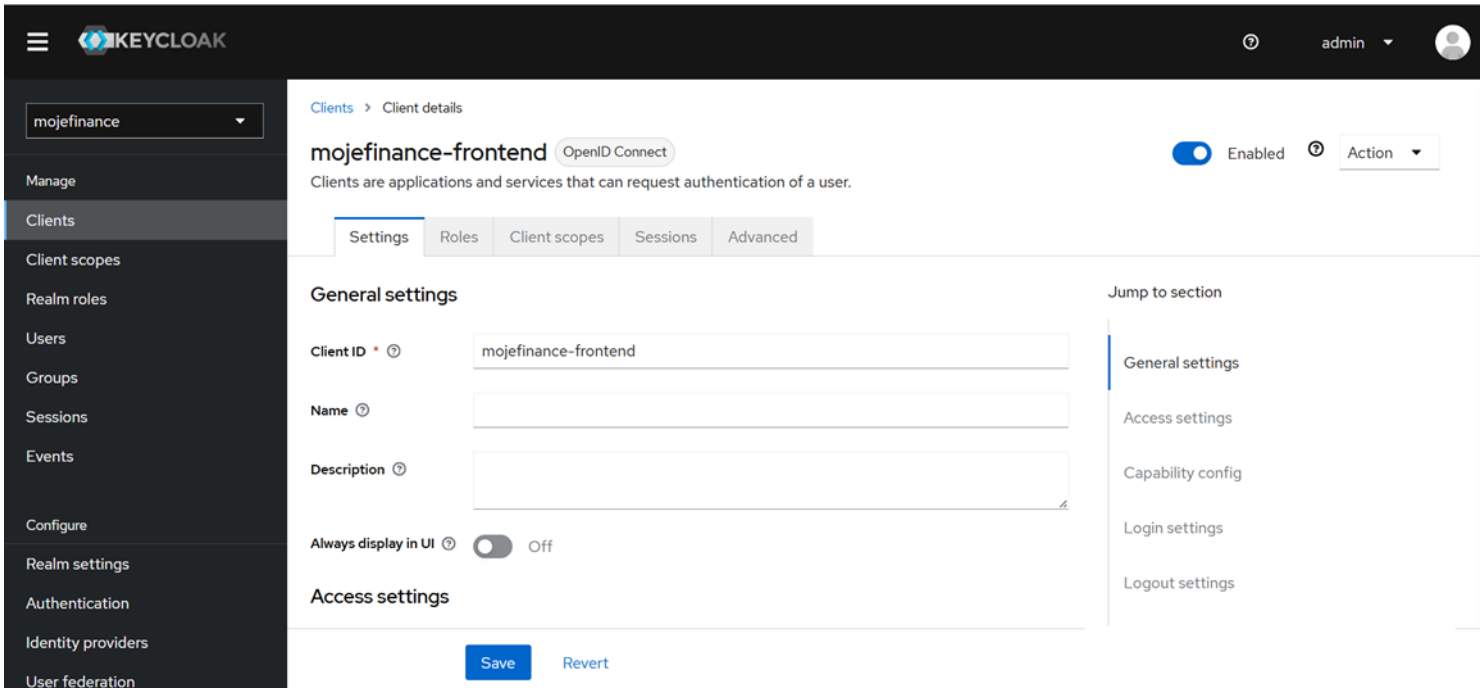

Figure 7.1. Keycloak Administrator Dashboard

7.2.2 Authorization Service

The Authorization Service module encapsulates the complex OAuth 2.0 flow required for banking integration. It uses the Spring Security framework to manage mutual TLS communications, execute authorization code exchanges, automate token rotation, and securely persist client credentials.

The core infrastructure is defined within the `Oauth2ClientManagerConfig` class. Institutions like Komerční banka and Česká spořitelna require specific mTLS certificates for secure communication. The configuration establishes specialized `RestTemplate` instances bound to explicit SSL bundles for each institution.

```
public RestTemplate baseSslRestTemplate(String bundleName) {
    RestTemplate restTemplate = builder
            .setSslBundle(sslBundles.getBundle(bundleName))
            .build();

    restTemplate.setMessageConverters(getCustomMessageConverters());
    restTemplate
    .setErrorHandler(new OAuth2ErrorResponseErrorHandler());
    return restTemplate;
}
```

Listing 7.1. RestTemplate with SSL

These templates are then injected into customized `OAuth2AccessTokenResponseClient` implementations, ensuring that every outgoing token request utilizes the correct cryptographic identity and includes required proprietary headers, such as the specific API key headers.

```
@Bean
public OAuth2AccessTokenResponseClient
<OAuth2AuthorizationCodeGrantRequest>
authorizationCodeTokenResponseClient(
    @Qualifier("kbRestTemplate") RestTemplate kbTemplate,
    @Qualifier("csobRestTemplate") RestTemplate csobTemplate,
    @Qualifier("defaultRestTemplate") RestTemplate defaultTemplate)
{
    ...
}
```

Listing 7.2. Customized OAuth2AccessTokenResponseClient

When a user successfully completes the authentication on a bank's external portal, the institution redirects the user back to the application along with a temporary authorization code. The `ExchangeTokenHelper` class intercepts this callback. It programmatically constructs an `OAuth2AuthorizationCodeGrantRequest`, executes the exchange to retrieve the initial access token and refresh token, and delegates the payload to the storage layer.

```
@Override
public void connectAuthorizedClient(ConnectAuthorizedClientRequest
request) throws ClientRegistrationNotFoundException {
    exchangeTokenHelper.exchangeToken(
        request.getClientRegistrationId(),
        request.getCode()
    );
}
```

Listing 7.3. Use of ExchangeTokenHelper

7.2.2.1 Token Storage and Encryption

Financial access tokens provide direct programmatic access to sensitive banking data and require strict protection at rest. Spring Security provides a standard `JdbcOAuth2AuthorizedClientService` to persist these OAuth2 entities within a relational database. To prevent credential exposure in the event of a database compromise, the application extends this capability by implementing a custom `AuthorizedClientService` decorator.

```
@Bean
public OAuth2AuthorizedClientService authorizedClientService(
    JdbcTemplate jdbcTemplate,
    ClientRegistrationRepository clientRegistrationRepository,
    @Value("${security.oauth2.encryption.password}") String password,
    @Value("${security.oauth2.encryption.salt}") String salt) {

    JdbcOAuth2AuthorizedClientService jdbcService =
        new JdbcOAuth2AuthorizedClientService(
            jdbcTemplate,
            clientRegistrationRepository
        );

    return new AuthorizedClientService(jdbcService, password, salt);
}
```

Listing 7.4. Token Storage

This custom service wraps the default JDBC implementation. It intercepts the `OAuth2AuthorizedClient` object immediately before database persistence and applies symmetric encryption to the token strings. The encryption utilizes a configured secret password and salt via Spring Crypto's `TextEncryptor`. When the system later retrieves the client, the service automatically decrypts the values before returning them to the application logic.

```
@Override
public <T extends OAuth2AuthorizedClient> T loadAuthorizedClient
(String clientRegistrationId, String principalName) {
    OAuth2AuthorizedClient authorizedClient = delegate.
    loadAuthorizedClient(clientRegistrationId, principalName);
    if (authorizedClient == null) {
        return null;
    }
    return (T) decrypt(authorizedClient);
}

@Override
public void saveAuthorizedClient
(OAuth2AuthorizedClient authorizedClient, Authentication principal) {
    delegate.saveAuthorizedClient(encrypt(authorizedClient), principal);
}
```

Listing 7.5. Token Encryption and Decryption

7.2.2.2 Token Rotation

Bank access tokens typically possess short lifespans, often expiring within short time frame. The system automates the token rotation process to maintain persistent bank connections without requiring repeated user intervention.

This automation is managed by the `DefaultOAuth2AuthorizedClientManager`, configured with an `OAuth2AuthorizedClientProvider`. When a business module (such as the Product Service) requires an access token to fetch new transaction data, it invokes the `authorizeClient` method. The manager retrieves the encrypted token from the database, decrypts it, and evaluates its expiration timestamp.

```
@Bean
public OAuth2AuthorizedClientManager authorizedClientManager(
    ClientRegistrationRepository clientRegistrationRepository,
    OAuth2AuthorizedClientRepository authorizedClientRepository,
    OAuth2AccessTokenResponseClient<OAuth2RefreshTokenGrantRequest>
    refreshTokenTokenResponseClient) {

    OAuth2AuthorizedClientProvider authorizedClientProvider =
            OAuth2AuthorizedClientProviderBuilder.builder()
                    .authorizationCode()
                    .refreshToken(
                        configurer -> configurer
                            .accessTokenResponseClient(
                                refreshTokenTokenResponseClient)
                    )
                    .build();

    DefaultOAuth2AuthorizedClientManager manager =
    new DefaultOAuth2AuthorizedClientManager(
            clientRegistrationRepository,
            authorizedClientRepository);
    manager.setAuthorizedClientProvider(authorizedClientProvider);
    return manager;
}
```

Listing 7.6. Token Rotation

If the access token is expired, the manager automatically halts the outgoing request. It constructs an `OAuth2RefreshTokenGrantRequest`, securely transmits the refresh token to the specific bank's authorization server using the pre-configured mTLS client, and acquires a new access token. The system then updates the encrypted database records with the new token values and returns the valid token to the calling service, allowing the data retrieval process to proceed without manual disruption.

7.3 Features

This section outlines the implementation of the primary user-facing functionalities. It details how the backend services process, aggregate, and deliver financial data to fulfill the core use cases of the application.

7.3.1 Dashboard

The dashboard serves as the central analytical hub for the user. When the client application requests the dashboard view, the backend synchronously aggregates data across all linked bank connections to construct a snapshot of the user's current financial standing.

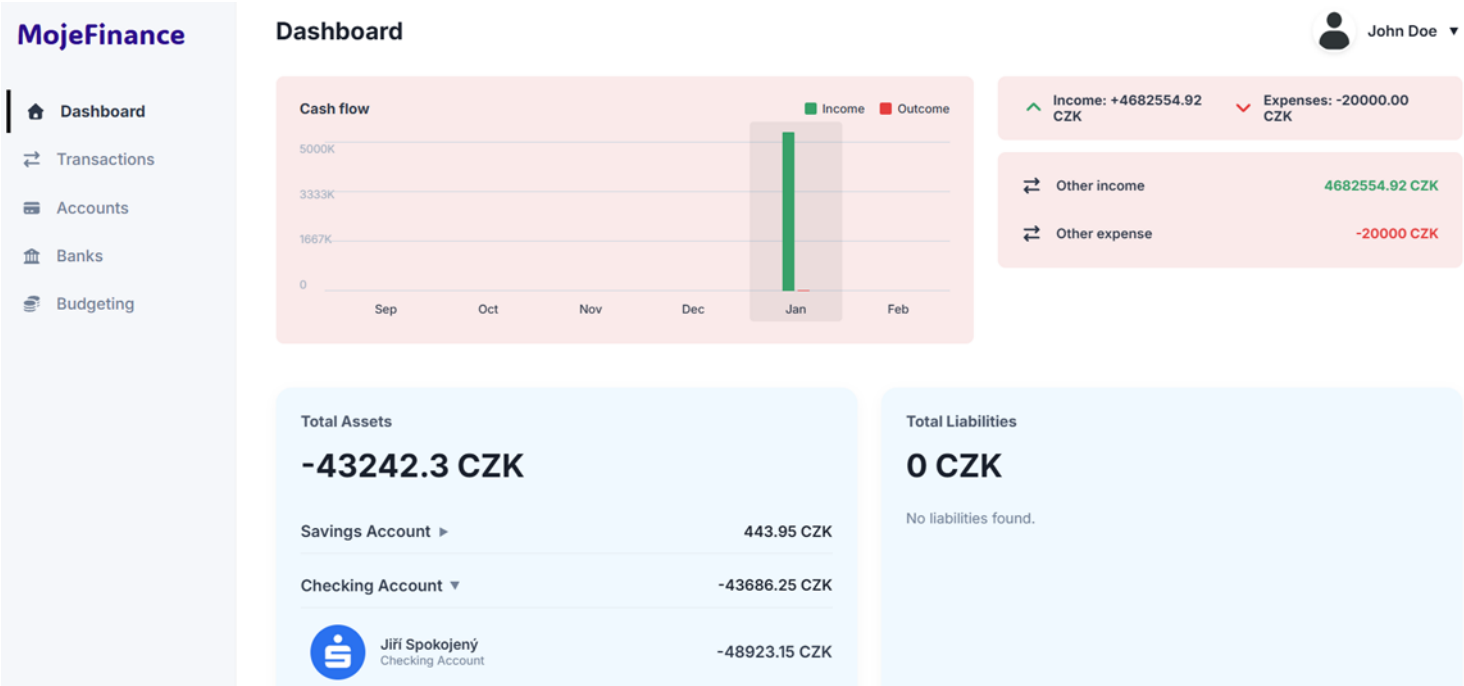

Figure 7.2. Feature — Dashboard

7.3.1.1 Cash Flow Summary

The cash flow summary visualizes financial movements from all linked accounts over specific periods. To support this feature efficiently, the backend `Product Service` utilizes the `TransactionsGroupingHelper` to systematically aggregate the data.

Transactions are grouped by their booking month and year and then by category. Within each group, the system calculates the sum of all incoming funds and the sum of all outgoing expenses. Performing this aggregation on the server side significantly reduces the data payload transmitted over the network and completely offloads complex grouping calculations from the frontend application.

7.3.1.2 Assets and Liabilities

To provide an accurate calculation of the user's net worth, the system must logically distinguish between positive liquid balances and outstanding debts. The backend processes the user's entire financial portfolio by evaluating the `ProductCategory` enumeration assigned to each account.

Standard checking accounts, savings, and investments are automatically classified as assets, whereas credit cards, mortgages, and personal loans are categorized as liabilities. The service tier calculates the cumulative totals for both categories. The resulting `AssetsAndLiabilitiesResponse` data structure allows the dashboard interface to clearly display the user's total available assets against their total liabilities.

7.3.2 Banks Overview

The Banks Overview feature serves as the primary management interface for external financial integrations, exposing its lifecycle operations through the `BankController`.

7.3.2.1 Data Retrieval

When the client sends `GET` request to the `/api/banks` endpoint, the `BankConnectionService` retrieves all registered integrations. It maps these internal entities to

a standardized response, explicitly detailing their connection status (e.g., `CONNECTED` or `DISCONNECTED`) to indicate the validity of the underlying access tokens.

```
private void updateBankConnectionStatus(
    List<BankConnection> connectedBanks,
    String principalName
) {
    for (BankConnection connectedBank : connectedBanks) {
        String clientRegistrationId = connectedBank
        .getClientRegistrationId();

        boolean authorizedClientDoesNotExist =
        authorizedClientDoesNotExist(
            principalName,
            clientRegistrationId
        );
        if (authorizedClientDoesNotExist) {
            connectedBank.setBankConnectionStatus(
                BankConnectionStatus.DISCONNECTED
            );
            BankConnectionEntity bankEntityToUpdate =
            bankConnectionDomainMapper
            .toBankConnectionEntity(connectedBank);
            bankConnectionRepository
            .updateConnectedBank(bankEntityToUpdate);
        }
    }
}
```

Listing 7.7. Update of Connection Status

7.3.2.2 Connection Initialization

After the user authenticates on the target bank's portal, the institution redirects back to the application with a temporary authorization code.

The frontend transmits this code to the backend via a `POST` request to `/api/banks/connect`. The backend then delegates the token exchange to the Authorization Service and persists the active `BankConnection` entity in the database.

7.3.2.3 Disconnection

Users can terminate an existing bank connection at any time via a `DELETE` request to the `/api/banks/disconnect/bankId` endpoint.

The backend proactively interfaces with the token storage layer to permanently remove all associated OAuth2 materials. This deletion process ensures that the system retains no unauthorized access or refresh tokens linked to that specific institution.

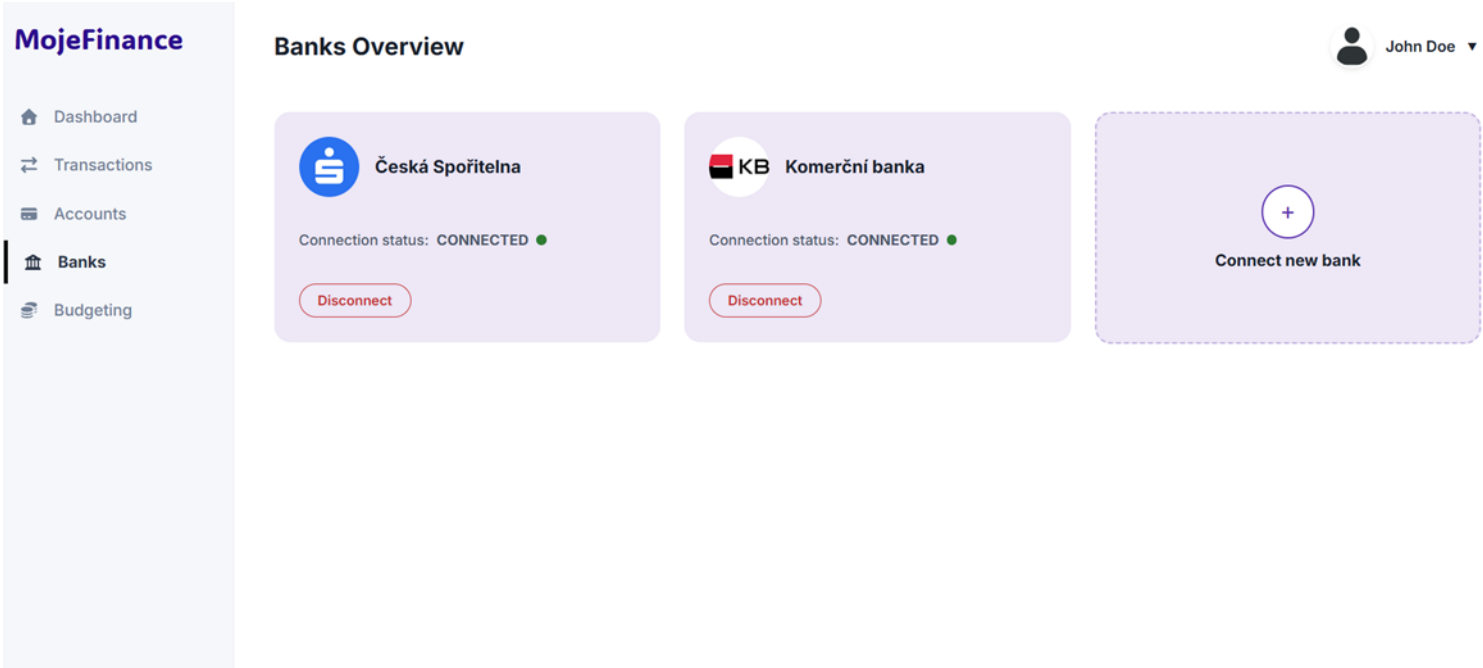

Figure 7.3. Feature — Banks Overview

7.3.3 Accounts Overview

The Accounts Overview feature provides a view of the user's financial products across all linked institutions. The client application retrieves this data by issuing a `GET` request to the `/api/products` endpoint. Then the `ProductService` retrieves all active bank connections associated with the user. For each valid connection, the service dynamically retrieves the required access tokens and executes real-time HTTP requests to the respective banking APIs.

To manage these external network communications, the backend uses Spring Cloud OpenFeign. By annotating methods with standard Spring web bindings, OpenFeign automatically generates the proxy implementations required to execute the requests.

Below is an example of the OpenFeign client used to interface with the Komerční banka API:

```
@FeignClient(
    name = "KBApiFeignClient",
    url = "${external.api.kb.base-url}",
    configuration = KBFeignConfig.class
)
public interface KBApiFeignClient {

    @GetMapping(value = "${external.api.kb.accounts-path}",
    consumes = MediaType.APPLICATION_JSON_VALUE)
    ResponseEntity<GetAccountListResponse> getAccounts(
            @RequestHeader("Authorization") String authorization
    );

    @GetMapping(value = "${external.api.kb.balances-path}",
    consumes = MediaType.APPLICATION_JSON_VALUE)
    ResponseEntity<GetAccountBalanceResponse> getAccountBalance(
            @RequestHeader("Authorization") String authorization,
            @PathVariable String id
    );

    @GetMapping(value = "${external.api.kb.transactions-path}",
    consumes = MediaType.APPLICATION_JSON_VALUE)
    ResponseEntity<GeAccountTransactionsResponse> getTransactions(
            @RequestHeader("Authorization") String authorization,
            @PathVariable String id,
            @RequestParam("fromDate") String fromDate,
            @RequestParam("toDate") String toDate
    );
}
```

Listing 7.8. OpenFeign client for Komerční banka API integration

Once the raw data is retrieved, the backend must normalize the different JSON responses from the various institutions into a standardized `Product` data transfer object. This complex mapping process is implemented using the MapStruct library. MapStruct automatically generates high-performance, type-safe mapping code at compile time

```
@Mapper(componentModel = "spring")
public interface KBApiMapper extends TransactionsApiMapper {
    @Mapping(target = "products", source = "accounts")
    ProductsResponse toProductsResponse(GetAccountListResponse
    getAccountListResponse, @Context BankDetails bankDetails);

    @Mapping(target = "productId", source = "id")
    @Mapping(target = "productIdentification.iban", source =
    "identification.iban")
    @Mapping(target = "productIdentification.productNumber", source =
    "identification.other")
    @Mapping(target = "accountName", source = "nameI18N")
    @Mapping(target = "productName", source = "productI18N")
    @Mapping(target = "manuallyCreated", constant = "false")
    @Mapping(target = "bankCode", source = "servicer.bankCode")
    @Mapping(target = "bankDetails", expression = "java(bankDetails)")
    Product toDomainProduct(Account account, @Context BankDetails bankDe-
tails);

    //Other mapper methods
    ...
}
```

Listing 7.9. MapStruct Example

After normalization, the backend transmits the aggregated list to the client. The frontend implements client-side filtering. This allows users to dynamically isolate and view accounts associated with a specific banking institution without triggering additional network requests to the backend server.

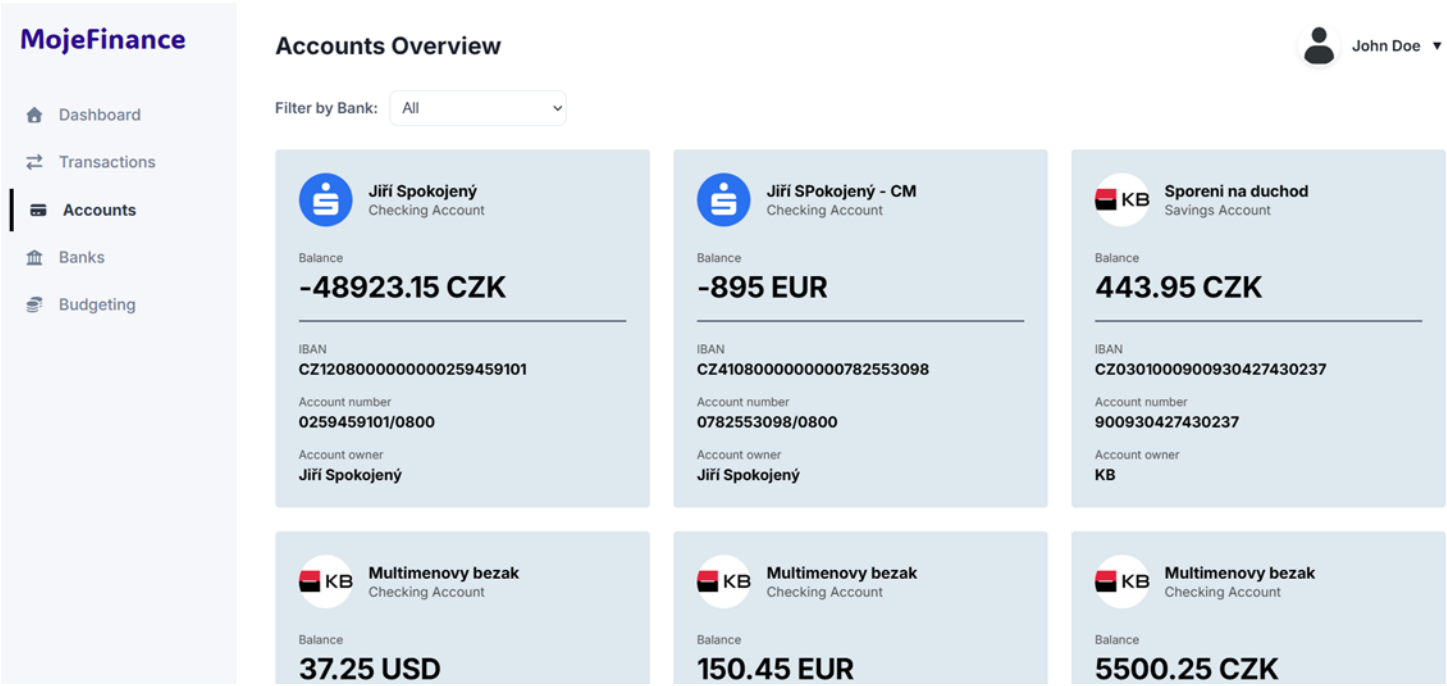

Figure 7.4. Feature — Accounts Overview

7.3.4 Transactions Overview

The Transactions Overview feature provides a chronological view of the user's financial movements. The backend exposes a dedicated REST endpoint (`/api/products/clientRegistrationId/accountId/transactions`) that accepts temporal parameters (`fromDate` and `toDate`) to fetch specific transaction histories from the external banking APIs.

To optimize network performance and reduce client-side processing, the backend processes the data, filtering out pending transactions and grouping the completed ones hierarchically by their booking month and structural category. Raw data retrieval works the same way as described in the 7.3.3 section. The aggregated payload is then transmitted to the client.

The frontend application consumes this structured data to render the transactions view, enabling client-side filtering between income and expenses. The frontend uses this grouped payload to power a dynamic cash flow chart. As the user switches between different accounts or adjusts the date filters, this interactive chart updates in real-time, providing an immediate visual summary of monthly spending habits directly alongside the detailed transaction list.

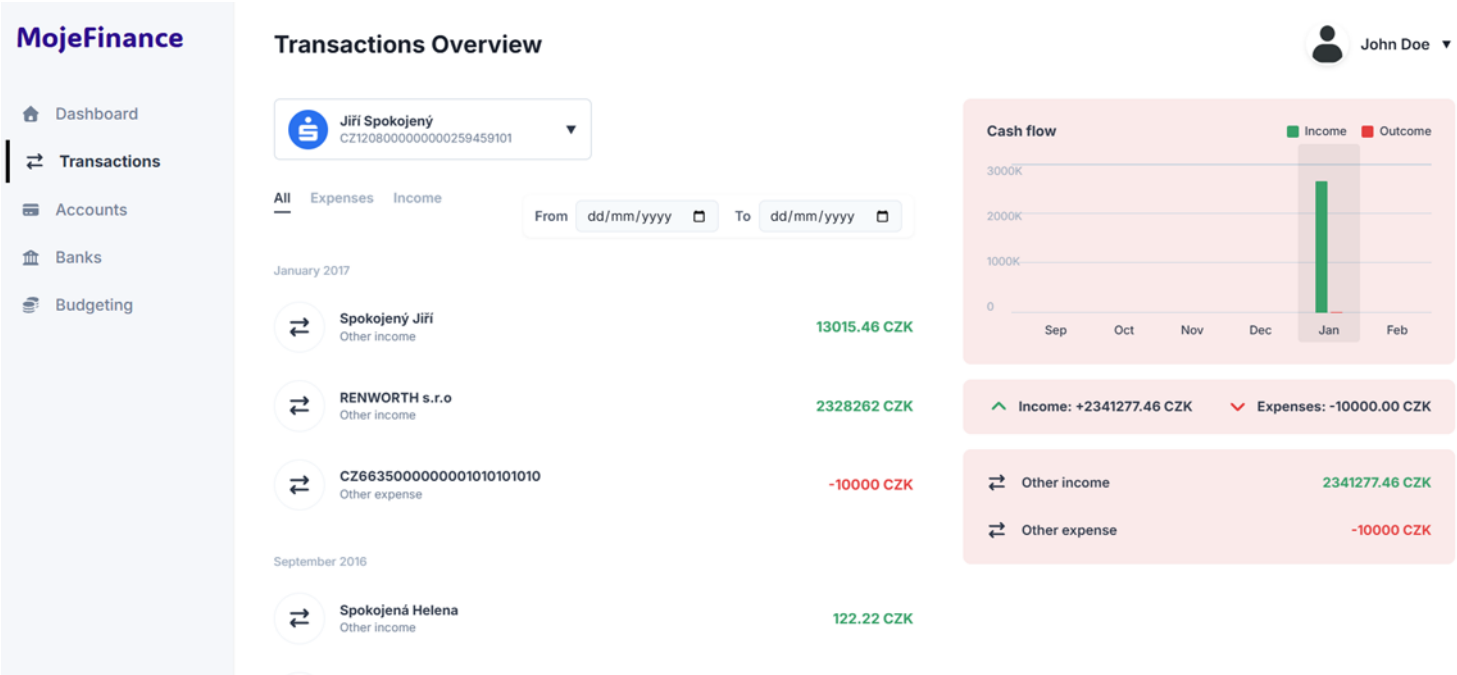

Figure 7.5. Feature — Transactions Overview

7.3.5 Budgeting

The Budgeting feature enables users to establish and monitor strict financial limits for specific transaction categories. The backend exposes a standard set of RESTful endpoints (`/api/budgets`) to handle the creation, modification, retrieval, and deletion of budgets.

When creating a budget, the user can define a specific start date. This date acts as a recurring anchor; the backend automatically resets the budget cycle on this exact day every subsequent month. If the frontend payload omits this parameter, the system

automatically injects the first day of the current month as the default fallback. The backend uses this timestamp to define the exact temporal window for evaluating the user's categorized expenses.

```
public void validateStartDate(Budget budget) {
    if (budget.getStartDate() == null) {
        LocalDate firstDayOfCurrentMonth = LocalDate.now()
        .withDayOfMonth(1);
        budget.setStartDate(firstDayOfCurrentMonth);
        log.info("Start date not provided. Setting start date to
        first day of current month: {}", firstDayOfCurrentMonth);
    }
}

public void updateBudgetStartDate(BudgetsResponse budgetsResponse) {
    for (Budget budget : budgetsResponse.getBudgets()) {
        YearMonth nextMonth = YearMonth
        .from(budget.getStartDate()).plusMonths(1);
        MonthDay monthDay = MonthDay.from(budget.getStartDate());
        LocalDate endDate = LocalDate.of(
            nextMonth.getYear(),
            nextMonth.getMonth(),
            monthDay.getDayOfMonth()
        );

        if (LocalDate.now().equals(endDate)) {
            log.info("Budget restarted for category: {}.
            Old start date: {}, New start date: {}",
            budget.getCategory(), budget.getStartDate(), endDate);
            budget.setStartDate(endDate);
            budget.setBudgetStatus(BudgetStatus.ACTIVE);
        }
    }
}
```

Listing 7.10. Start Date Processing

To provide accurate and real-time tracking, the backend service continuously evaluates the user's categorized expenses against these defined limits. If the calculated expenses for a specific category exceed the defined limit, the backend automatically updates the `budgetStatus` flag to `EXCEEDED`.

```java
private void enrichBudgets(List<Budget> budgetsForDate, Map<String,
    BigDecimal> spentAmountPerCategory)
{
    budgetsForDate.forEach(budget -> {
        BigDecimal spentAmount = spentAmountPerCategory
        .getOrDefault(
            budget.getCategory().getDisplayName(),
            BigDecimal.ZERO
        );
        Amount spentAmountObj = Amount.builder()
                .value(spentAmount)
                .currency(budget.getAmount().getCurrency())
                .build();
        budget.setSpentAmount(spentAmountObj);

        if (spentAmount.compareTo(budget.getAmount().getValue()) >= 0) {
            budget.setBudgetStatus(BudgetStatus.EXCEEDED);
            log.info("Budget exceeded for category: {}.
            Spent amount: {}, Budget amount: {}",
            budget.getCategory(), spentAmount,
            budget.getAmount().getValue());
        } else {
            budget.setBudgetStatus(BudgetStatus.ACTIVE);
            log.info("Budget active for category: {}.
            Spent amount: {}, Budget amount: {}",
            budget.getCategory(), spentAmount,
            budget.getAmount().getValue());
        }

        log.info("Budget category: {}, Spent amount: {}",
        budget.getCategory(), spentAmount);
    });
}
```

Listing 7.11. Processing User's Categorized Expenses

The frontend application consumes this payload to render a budgeting dashboard. It provides an interface for users to manage their limits and uses the calculated data to display progress bars for each tracked category. To ensure user awareness, the interface emphasizes exceeded budgets by automatically sorting them to the top of the layout, applying visual warning tags, and triggering a global alert banner across the top of the view.

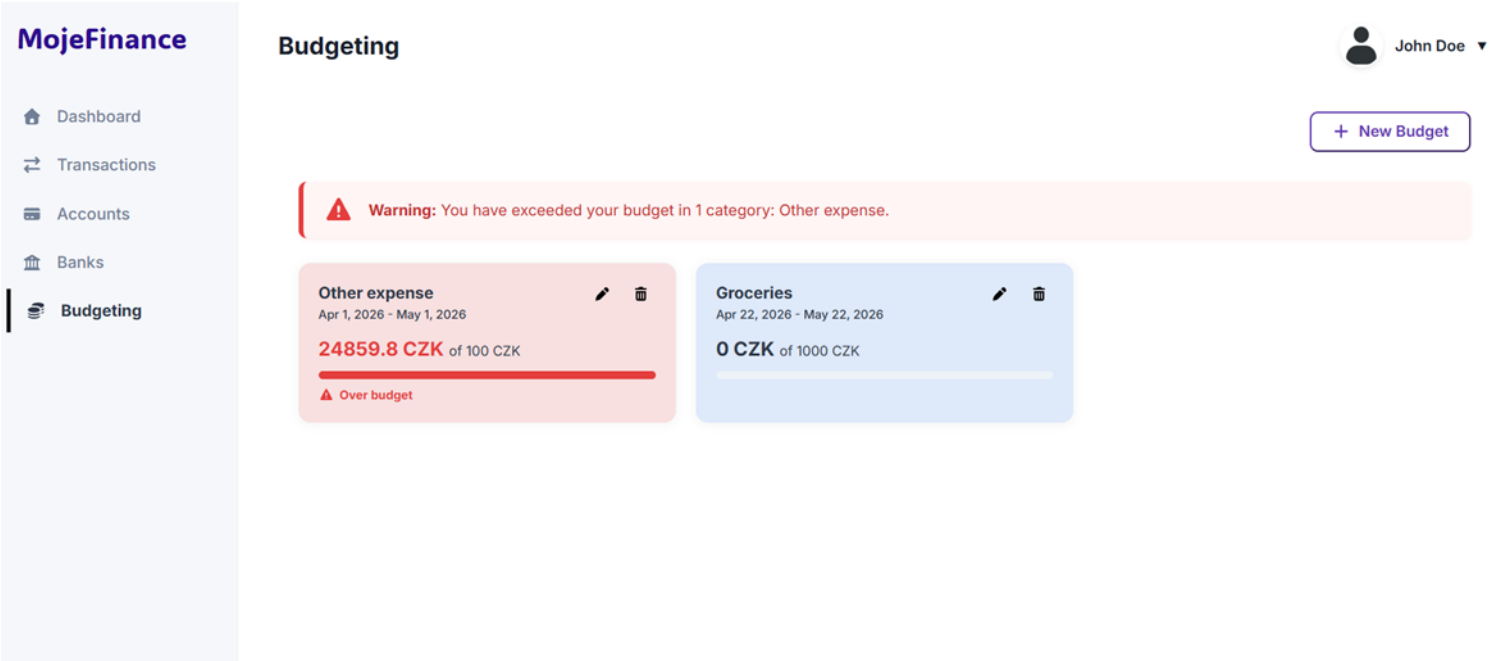

Figure 7.6. Feature — Budgeting

7.3.6 Categorization Service

The Categorization Service is a backend component that enriches raw financial data by assigning transactions and products to standardized categories.

Instead of relying on rigid, hard-coded rules or complex text matching, the implementation utilizes a highly efficient hybrid approach: a self-updating database dictionary paired with an integrated Google Gemini AI provider.

When the system receives data during synchronization, the service extracts the counterparty name and transaction direction from every transaction and the product name from every product. The extracted names are queried against the `TransactionMappingEntity` table in the database. If the system has seen this mapping before, it instantly retrieves the saved category.

Any names that are completely new and not found in the dictionary are sent to the Gemini AI API, which evaluates the names and returns the most probable categories from the system's predefined options. The newly acquired AI answers are saved back into the database mapping tables. This ensures the system continuously learns, which speeds up processing times and minimizes the need for external AI API calls.

```java
public Map<String, TransactionCategory> saveNewTransactionMappings
(Set<String> unmappedTransactions)
{
    Map<String, TransactionCategory> transactionMappings =
    new HashMap<>();

    for (String unmappedTransactionName : unmappedTransactions) {
        String prompt = buildPrompt(unmappedTransactionName);
        String geminiResponse = geminiProvider.askGemini(prompt);
        TransactionCategory mappedCategory = getTransactionCategory(
            unmappedTransactionName,
            geminiResponse
        );

        // Saving new mapping into the database
        ...

        transactionMappings
        .put(unmappedTransactionName, mappedCategory);
    }
    return transactionMappings;
}

@Override
public String askGemini(String prompt) {
    log.info("Asking Gemini with prompt: {}", prompt);

    GenerateContentResponse response;
    try {
        Client client = Client.builder()
                .apiKey(geminiApiKey)
                .build();

        response = getModels(client).generateContent(
                GEMINI_MODEL_NAME,
                prompt,
                null
        );
    } catch (Exception e) {
        log.error("Error while asking Gemini.", e);
        return EMPTY_STRING;
    }
    log.info("Received response from Gemini: {}", response.text());
    return Objects.requireNonNull(response.text()).trim();
}
```

Listing 7.12. AI-driven Categorization

7.3.7 Currency Exchange

When users connect bank accounts that hold foreign currencies (such as EUR or USD), directly summing these values with the local base currency (CZK) would result in mathematically incorrect totals for assets, liabilities, and cash flow charts.

To solve this, the backend implements a dedicated `CurrencyExchangeService`. This service intercepts foreign currency amounts and normalizes them into CZK before they are passed to the frontend or aggregated into dashboard totals.

```
public void exchangeProductBalances(List<Product> products) {
    for (Product product : products) {
        Amount originalAmount = product.getBalance();
        if (!CZK_CURRENCY_CODE.equals(originalAmount.getCurrency())) {
            Amount amountInCZK = currencyExchangeService
            .exchangeAmount(originalAmount);
            product.setBalance(amountInCZK);
        }
    }
}

@Override
public Amount exchangeAmount(Amount amount) {
    //Amount validation
    ...

    BigDecimal exchangeRate = externalApiProvider
    .getExchangeRates(amount.getCurrency());
    BigDecimal originalAmount = amount.getValue();
    BigDecimal result = originalAmount.multiply(exchangeRate);
    return Amount.builder()
            .value(result)
            .currency(CZK_CURRENCY_CODE)
            .build();
}
```

Listing 7.13. Currency Exchange

The system fetches daily exchange rate value using the Komerční banka Open API and caches it for 24 hours. Specifically, the service extracts the `cashBuy` rate for the requested foreign currency. This ensures the conversion reflects realistic banking market rates.

The conversion happens within the domain layer. The original foreign amount is multiplied by the fetched rate, generating a new `Amount` object with the `CZK` currency code.

7.4 Caching and Resilience

Integrating with multiple external banking APIs introduces inherent latency and reliability risks. Network timeouts, strict rate limits, or temporary outages on the provider's

side can severely degrade the user experience. To mitigate these risks and ensure high availability, the backend implements a dual strategy: data caching paired with fault-tolerance patterns using the Resilience4j library.

7.4.0.1 Caching Strategy

To minimize redundant network calls and accelerate data retrieval, the application utilizes Redis as a distributed cache.

The caching configuration is defined to allow different Time-To-Live policies depending on the volatility of the requested data. Standard banking data is cached for 30 minutes, whereas static daily metrics like exchange rates are cached for 24 hours.

```
@Configuration
@EnableCaching
public class RedisCacheConfig {

    @Bean
    public RedisCacheConfiguration cacheConfiguration() {
        return RedisCacheConfiguration.defaultCacheConfig()
            .entryTtl(Duration.ofMinutes(30))
            .disableCachingNullValues()
            // ... value serializer config
    }

    @Bean
    public RedisCacheManagerBuilderCustomizer
    redisCacheManagerBuilderCustomizer() {
        return (builder) -> builder
                .withCacheConfiguration("exchange-rates",
                        RedisCacheConfiguration.defaultCacheConfig()
                            .entryTtl(Duration.ofHours(24))
                            .disableCachingNullValues()
                            // ... value serializer config
    }
}
```

Listing 7.14. Redis Cache Configuration with variable TTLs

By annotating methods with `@Cacheable`, the system automatically intercepts requests and serves them directly from memory if the data is still valid.

```
@Override
//Resilience annotations
...
@Cacheable(value = "products", key = "#request.principalName + '-' +
request.bankDetails.clientRegistrationId")
public ProductsResponse getProducts(ProductsMessagingRequest request) {
//Products retrieving logic
...
}
```

Listing 7.15. Cacheable Annotation Example

7.4.0.2 Resilience and Circuit Breaking

When a cache miss occurs and an external API call must be made, the system protects itself by using Resilience4j's Retry and Circuit Breaker mechanisms.

```
@Override
@Retry(name = RESILIENCE_INSTANCE, fallbackMethod =
"fallbackGetProducts")
@CircuitBreaker(name = RESILIENCE_INSTANCE, fallbackMethod =
"fallbackGetProducts")
// Caching annotation
...
public ProductsResponse getProducts(ProductsMessagingRequest request) {
    //Products retrieving logic
    ...
}
```

Listing 7.16. Retry and Circuit Breaker Annotations Example

- **Retry Mechanism:** Temporary network glitches are handled by automatically retrying failed requests. The system attempts the call up to 3 times with a 2-second delay between attempts before giving up.
- **Circuit Breaker:** If a bank API experiences a sustained outage, repeated failures will trip the Circuit Breaker. Once the failure rate reaches 50 percent (evaluated over a sliding window of 10 calls), the circuit opens. Further requests to that specific API are immediately blocked for 15 seconds to prevent overwhelming the failing service and to save local application threads.

This behavior is centrally defined in the application's configuration file:

```yaml
resilience4j:
    circuitbreaker:
        instances:
            bankApi:
                slidingWindowSize: 10
                minimumNumberOfCalls: 5
                failureRateThreshold: 50
                waitDurationInOpenState: 15s
                permittedNumberOfCallsInHalfOpenState: 3
                automaticTransitionFromOpenToHalfOpenEnabled: true
    retry:
        instances:
            bankApi:
                maxAttempts: 3
                waitDuration: 2s
                retryExceptions:
                    - cvut.fel.sit.shared.exception.ServiceException
                    - feign.FeignException
```

Listing 7.17. Retry and Circuit Breaker configuration

When the circuit is open or retries are entirely exhausted, the system elegantly downgrades via explicitly defined fallback methods. Instead of throwing a critical HTTP 500 error to the frontend, these fallbacks intercept the failure and return safe, empty states.

```java
private ProductsResponse fallbackGetProducts(
    ProductsMessagingRequest request,
    Throwable throwable
) {
    log.error("CircuitBreaker/Retry Fallback triggered for getProducts.
    Bank: {}. Reason: {}", request.getBankDetails().getBankName(),
    throwable.getMessage());
    return ProductsResponse.builder()
        .products(Collections.emptyList())
        .build();
}
```

Listing 7.18. Fallback Method Example

This ensures that a single failing bank API does not crash the entire synchronization process, guaranteeing that the dashboard remains functional and allows the user to view their other unaffected accounts seamlessly.

7.5 Deployment

7.5.1 CI Pipeline

7.5.2 Docker

7.6 Error Handling

Chapter 8
Testing

8.1 Unit Testing

8.2 Integration Testing

8.3 End-to-end Testing

Chapter 9
Conclusion

This project demonstrates the design of a secure and user-friendly web application for managing personal bank accounts. Through analysis of existing finance platforms and Czech banking APIs, a solution was proposed that aggregates financial data, supports budgeting and visualization, and respects PSD2 and GDPR requirements. The chosen service-oriented architecture and technology stack ensure scalability, maintainability, and resilience. Although production access to most bank APIs is limited by regulatory constraints, the application remains functional through sandbox integrations and fictive accounts. The project provides a solid foundation for further development, including potential commercial deployment or extension toward advanced financial analytics.

References

Appendix A
Zadání práce

Develop a web-based application to help private users manage their multiple personal banking accounts. The application should allow users to monitor accounts from various Czech financial institutions. It should aggregate multiple accounts into a unified dashboard, allow tracking of transactions from each account, and support budgeting. Research current solutions and design the basic functionalities of the proposed application. The developed application should include functionality for displaying bank accounts, transactions, and budgets. Identify a suitable framework for the frontend and backend of your application, and justify your selection of technologies. Create a set of scenarios to verify the functionality of both individual parts and the overall application. Based on the results, evaluate the application's current state and suggest additional functionality or other improvements.

Appendix B
Zkratky a symboly

Tento text je až na výjimky převzat z [??].

B.1 Zkratky

Jako příklad pro popis zkratek poslouží pojmy ze světa TEXu.

- **TEX** Program na přípravu elektronické sazby vysoké kvality vytvořený Donaldem Knuthem. Program zahrnuje interpret makrojazyka. Název programu se vyslovuje „tech“.
- **LuaTEX** Rozšířený program TEX o možnost užití Unicode fontů a programovacího jazyka Lua.
- **METAFONT** Program a makro jazyk pro generování fontů z vektorového do bitmapového formátu vytvořený Donaldem Knuthem.
- **METAPOST** Program generující vektorovou grafiku založený na METAFONTu vytvořený Johnem Hobby.
- **plainTEX** Originální TEXový formát (rošíření na úrovni makrojazyka). Je součástí každé distribuce TEXu a je vytvořen Donaldem Knuthem.
- **OpTEX** TEXový formát rozšiřující plainTEX s využitím maker OPmac pro LuaTEX.
- **$\mathcal{CS}$plain** TEXový formát rozšiřující plainTEX o možnosti sazby v českém a slovenském jazyce vytvořený Petrem Olšákem.
- **LATEX** Nejznámější TEXový formát (rozšíření na úrovni makrojazyka) vytvořený Leslie Lamportem. Existuje obludné množství různých balíčků, které pomocí makrojazyka TEXu dále rozšiřují výchozí možnosti LATEXu. Rozličné uživatelské požadavky jsou nejčastěji řešeny použitím vhodného balíčku.
- **OPmac** Olšákova PlainTEXová makra nabízející uživatelům plainTEXu podobné možnosti, jako LATEX, ovšem přímočařeji a jednodušeji.
- **ConTEXt** Typografický systém vystavěný na LuaTEXu a na předpřipravených makro souborech vytvořený týmem v čele s Hansem Hagenem. Rozličné uživatelské požadavky jsou nastavovány pomocí přiřazení hodnot klíčovým slovům společně s možností TEXového, METAPOSTího a Lua programování.

B.2 Symboly

- π Konečná verze TEXu zmíněna v Knuthově TEXtamentu.
- e Konečná verze METAFONTu.
- 2ε Současná verze LATEXu používaná od roku 1994. Počítá se s ní jako s přechodnou verzí mezi původní Lamportovou verzí LATEX 2.09 a cílovou verzí LATEX 3. Tento přechodný stav už trvá 27 let.

B.3 Soubory, které jsou součástí CTUstyle3

`ctustyle3.tex`	TEXová makra implementující šablonu ve verzi 3 (pro OpTEX).
	Logo ČVUT podle[??] v modré a černé variantě.
`Technika-Regular.otf`	Metrika a kresby písma **Technika-Regular**.
`Technika-Italic.otf`	Metrika a kresby písma ***Technika-Italic***.
`Technika-Book.otf`	Metrika a kresby písma Technika-Book.
`Technika-BookItalic.otf`	Metrika a kresby písma *Technika-BookItalic*.
`Technika-Bold.otf`	Metrika a kresby písma **Technika-Bold**.
`Technika-BoldItalic.otf`	Metrika a kresby písma ***Technika-BoldItalic***.
`ctustyle-doc.tex`	Hlavní zdrojový soubor tohoto dokumentu.
`uvod.tex, popis.tex, prilohy.tex`	Zdrojové soubory čtené z `ctustyle-doc.tex` s jednotlivými kapitolami tohoto dokumentu.
`cmelak1.jpg`	Obrázek použitý v ukázce, jak vložit obrázek.
`mybase.bib`	Údaje použité pro generování seznamu literatury.
`ctustyle-doc.pdf`	Tento dokument.
`slides.tex`	Zdrojový text ilustrující použití CTUslides.
`slides.pdf`	Výstupní ukázka ilustrující použití CTUslides včetně návodu k použití.